



US012405867B2

(12) **United States Patent**
Ganigarakoppal Kantharaju et al.

(10) **Patent No.:** **US 12,405,867 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **CRASH-BASED RAID DATA STORAGE SYSTEM**

(56) **References Cited**

U.S. PATENT DOCUMENTS

- (71) Applicant: **Dell Products L.P.**, Round Rock, TX (US)
- (72) Inventors: **Nikhith Ganigarakoppal Kantharaju**, Hassan (IN); **Richa Kumari**, Bangalore (IN); **Abhijit Shashikant Mirajkar**, Bangalore (IN); **Ajay Sukumaran Nair Syamala Bai**, Bangalore (IN)
- (73) Assignee: **Dell Products L.P.**, Round Rock, TX (US)

5,548,711	A *	8/1996	Brant		G06F 11/2094
					714/E11.034
5,717,850	A *	2/1998	Apperley		G06F 11/2094
					714/48
9,996,477	B2	6/2018	Chan et al.		
2017/0123995	A1 *	5/2017	Freyensee		G06F 11/1076
2019/0332479	A1 *	10/2019	Gao		G06F 3/0689
2019/0391872	A1 *	12/2019	Kirkpatrick		G06F 11/1068
2020/0043524	A1 *	2/2020	Roberts		G06F 11/1088
2021/0034255	A1 *	2/2021	Vankamamidi		G06F 3/0631
2022/0066678	A1 *	3/2022	Singidi		G06F 3/064
2024/0085477	A1 *	3/2024	Turman		H04Q 9/00
2025/0036314	A1 *	1/2025	Ganigarakoppal Kantharaju		G06F 3/0659

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 63 days.

* cited by examiner

Primary Examiner — Matthew M Kim

Assistant Examiner — Matthew N Putaraksa

(74) *Attorney, Agent, or Firm* — Joseph Mencher

(57)

ABSTRACT

A crash-based RAID data storage system includes a chassis housing storage subsystems coupled to an operating system and a software RAID subsystem. During a runtime mode of the operating system, the software RAID subsystem reserves a RAID driver telemetry storage portion of a RAID logical storage subsystem provided by at least one storage device in the storage subsystems. In response to the operating system entering a crash dump mode from the runtime mode, the software RAID subsystem initializes respective controller devices in each of the storage subsystems, and during the crash dump mode of the operating system the software RAID subsystem stores RAID driver telemetry data generated by the software RAID subsystem in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device.

20 Claims, 31 Drawing Sheets

(21) Appl. No.: **18/229,293**

(22) Filed: **Aug. 2, 2023**

(65) **Prior Publication Data**

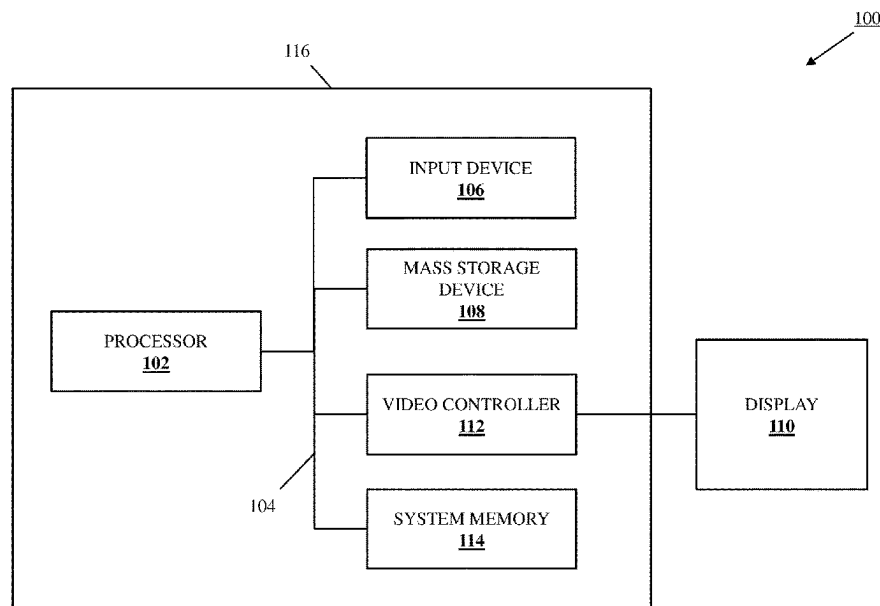
US 2025/0045177 A1 Feb. 6, 2025

(51) **Int. Cl.**
G06F 11/16 (2006.01)
G06F 12/02 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 11/1666** (2013.01); **G06F 12/023** (2013.01); **G06F 2201/805** (2013.01)

(58) **Field of Classification Search**
CPC G06F 11/1666; G06F 12/023; G06F 2201/805

See application file for complete search history.



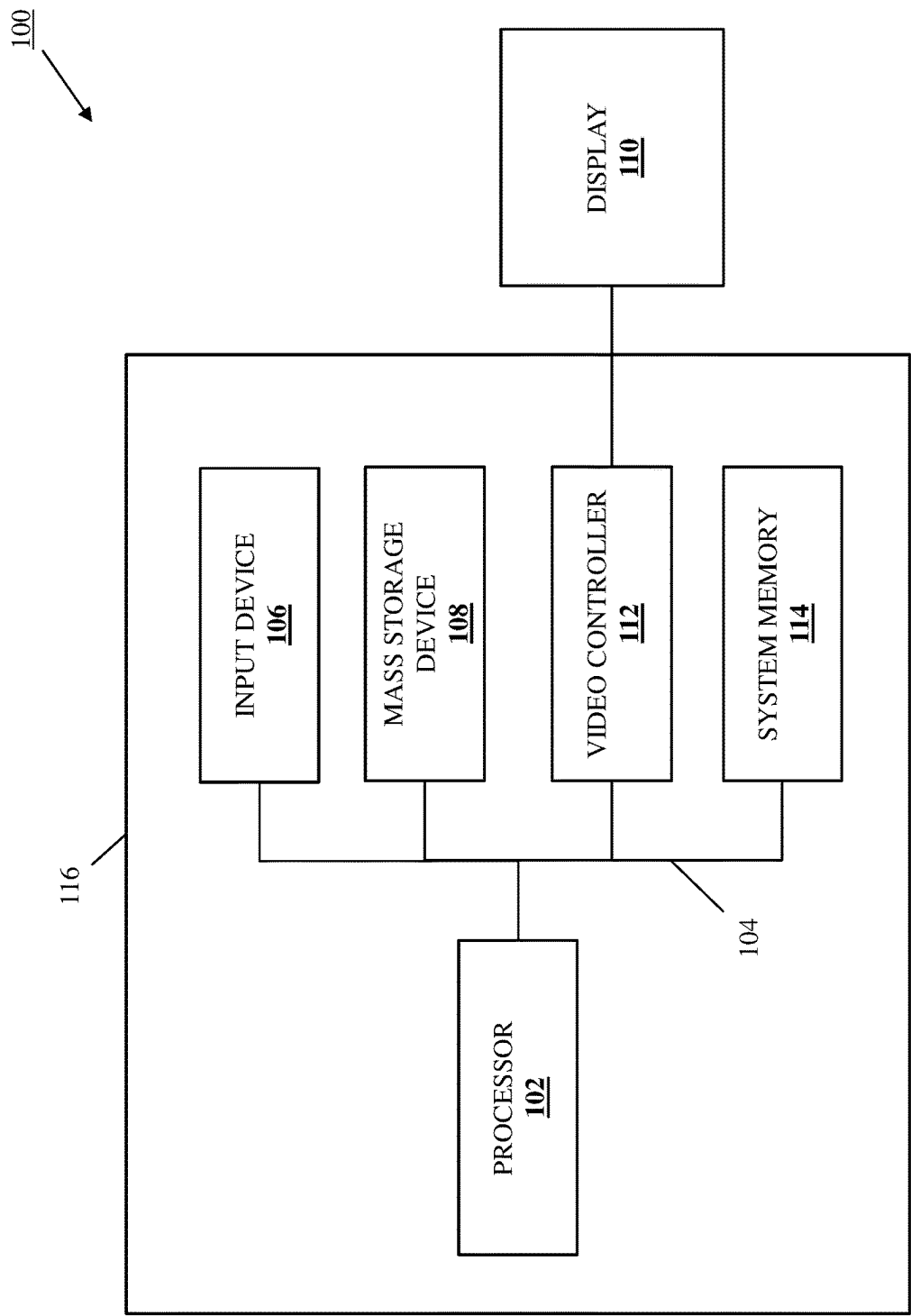


FIG. 1

202

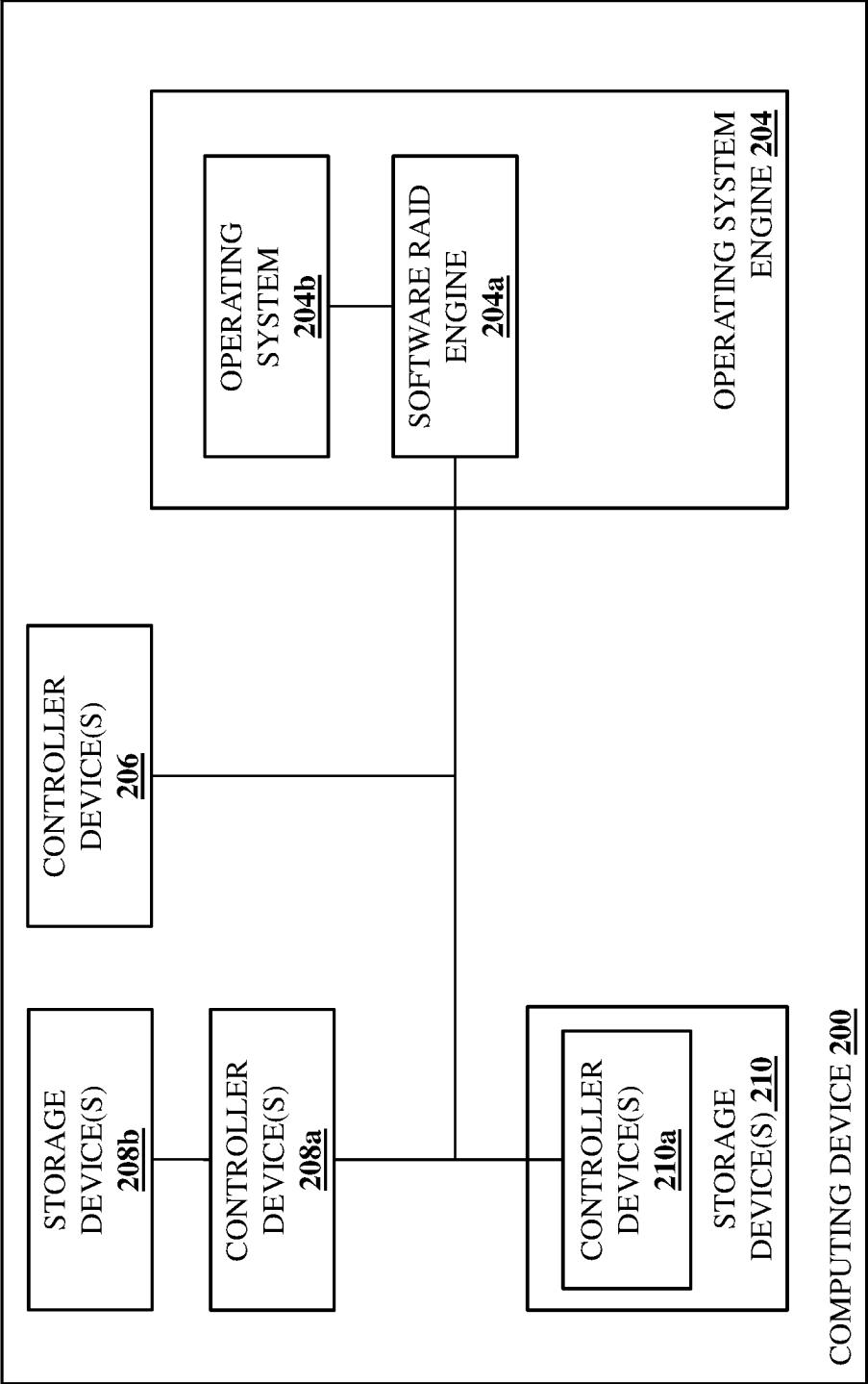
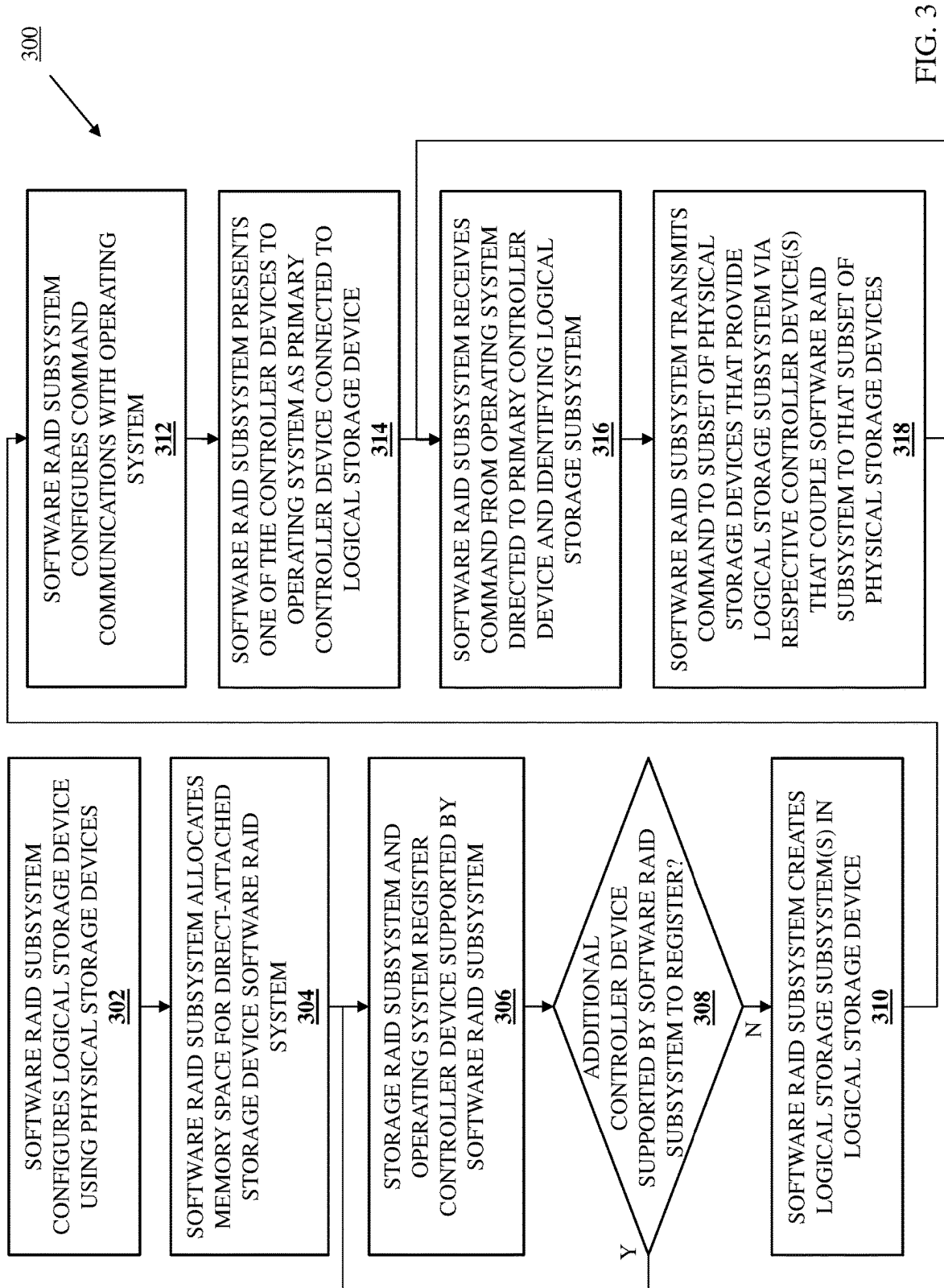


FIG. 2



202

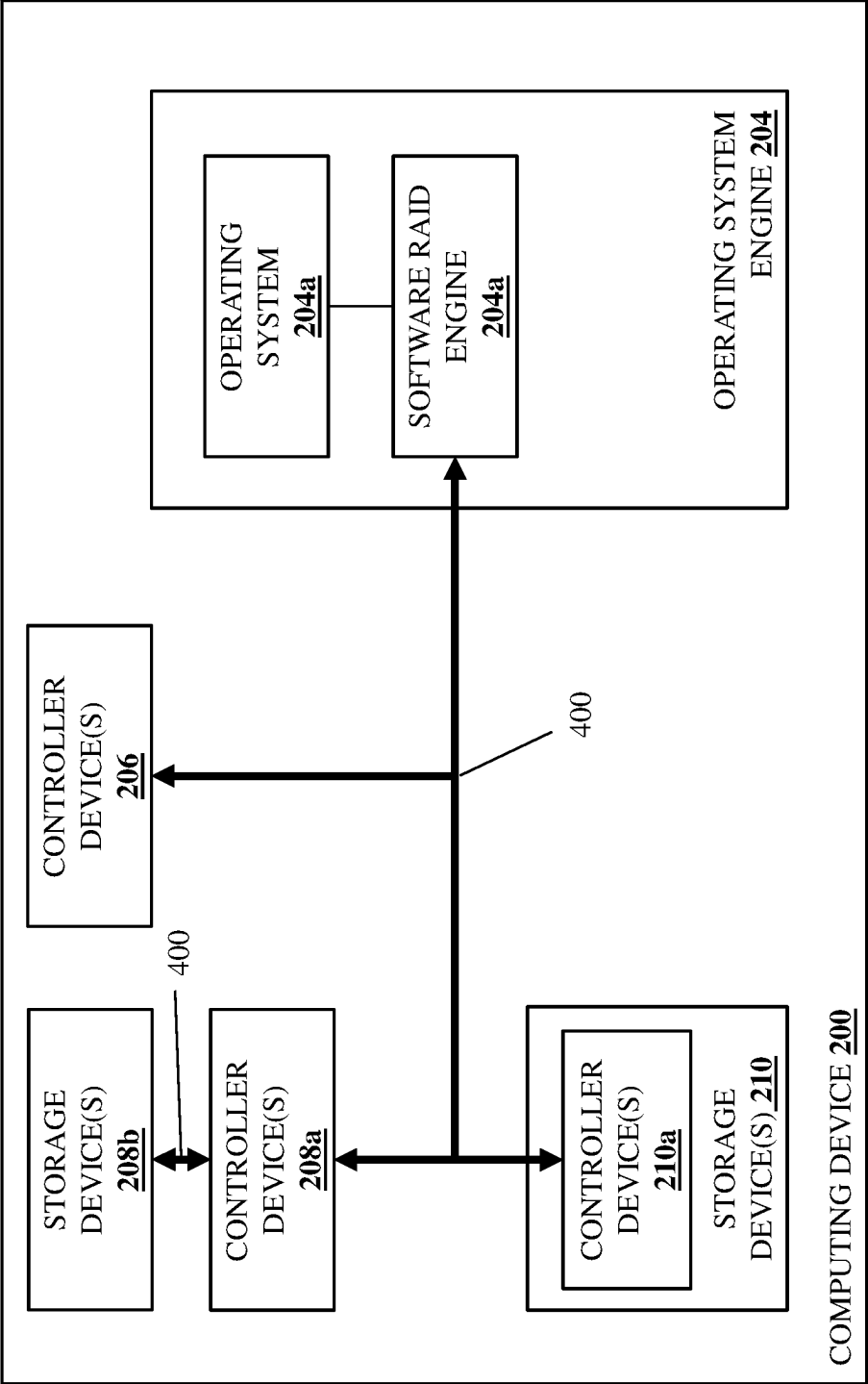


FIG. 4

202

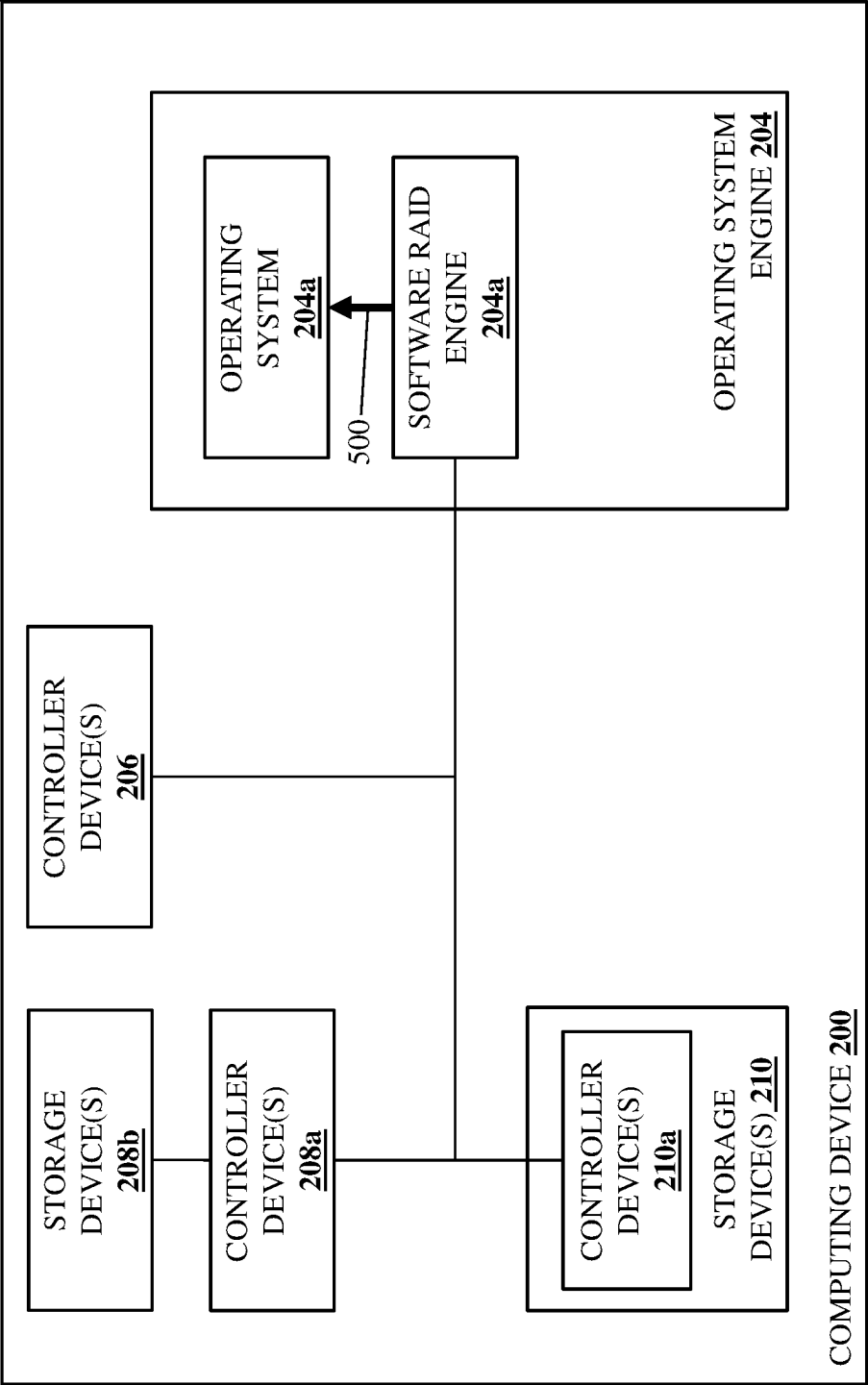


FIG. 5

202

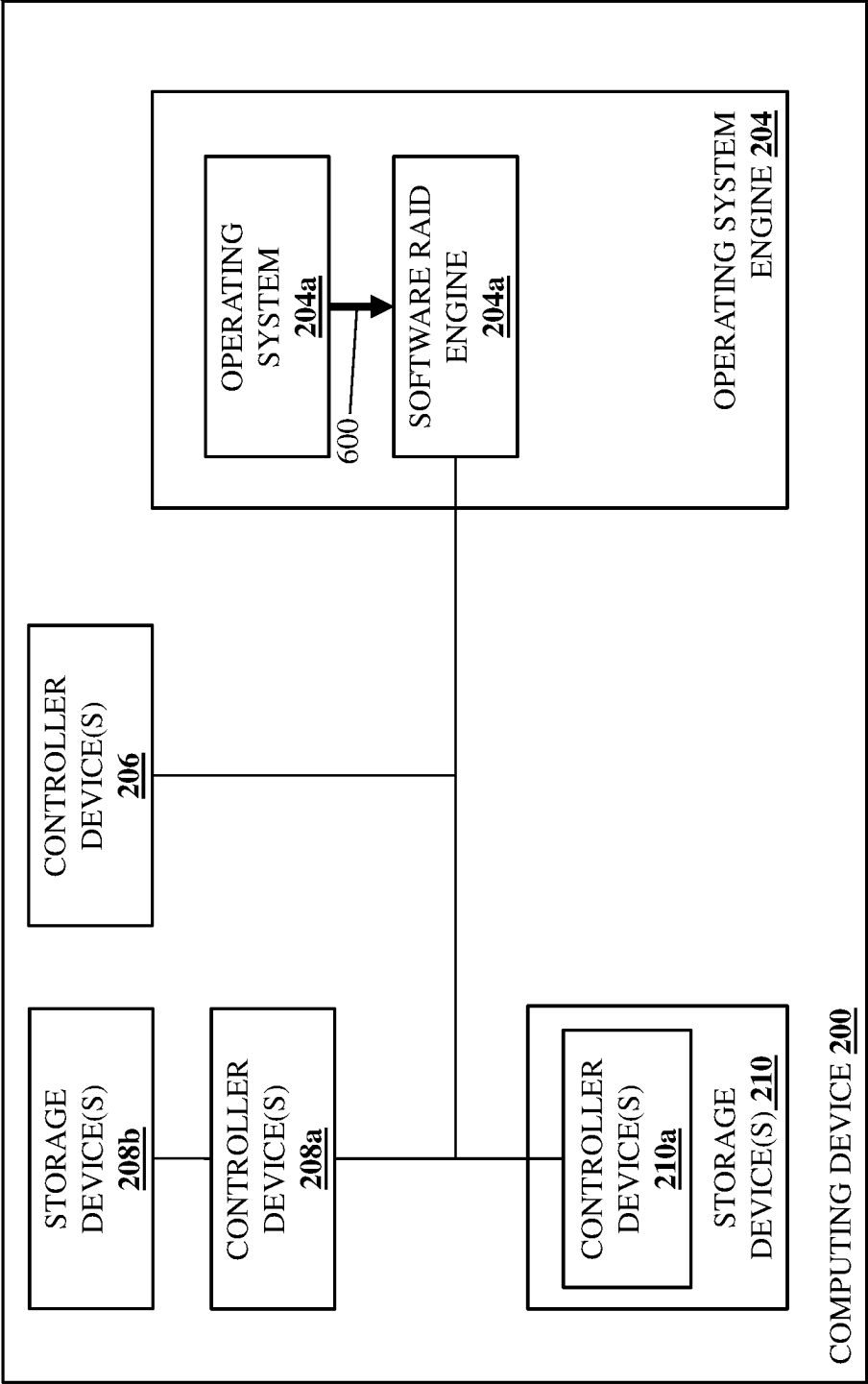


FIG. 6A

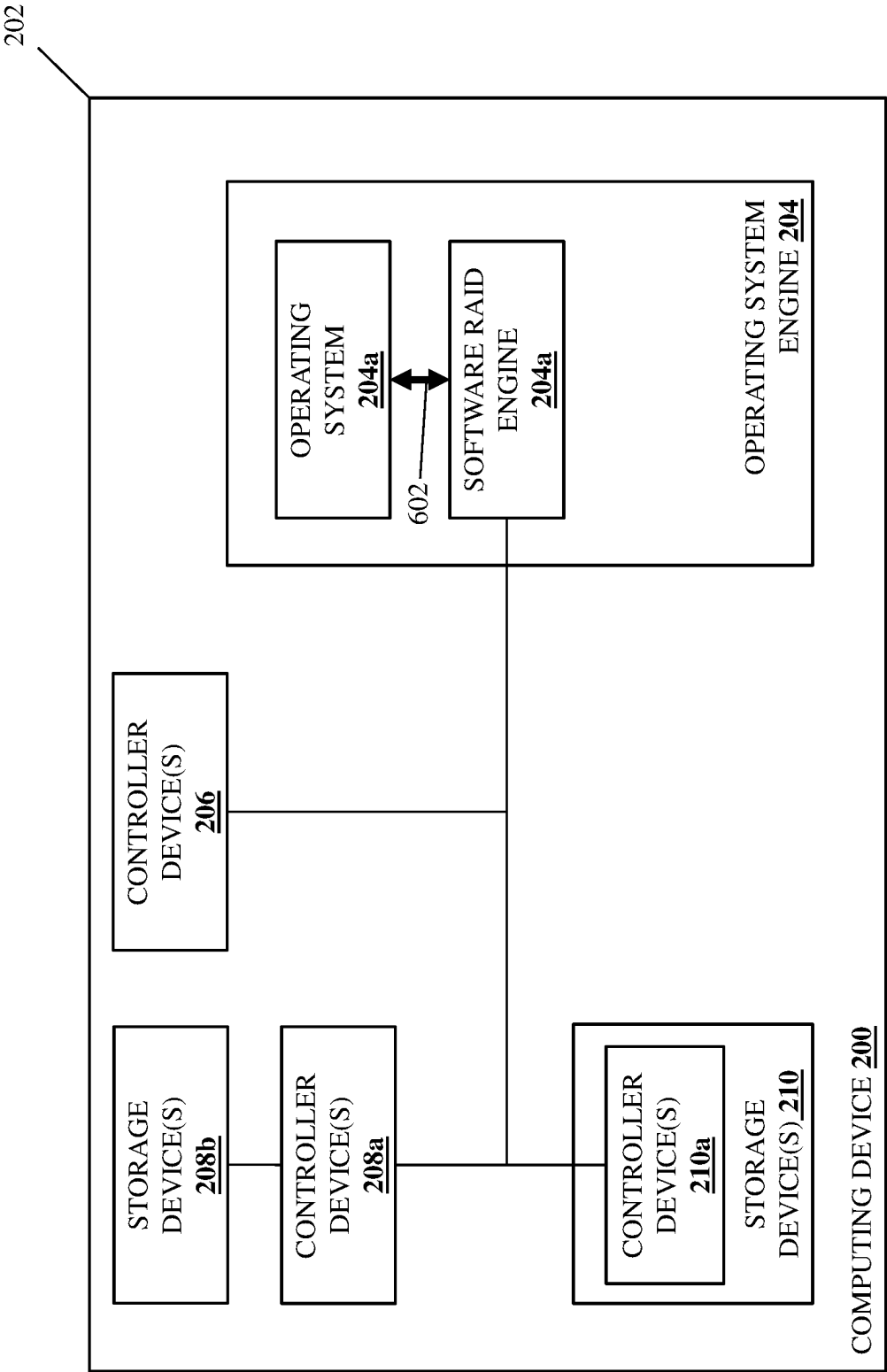


FIG. 6B

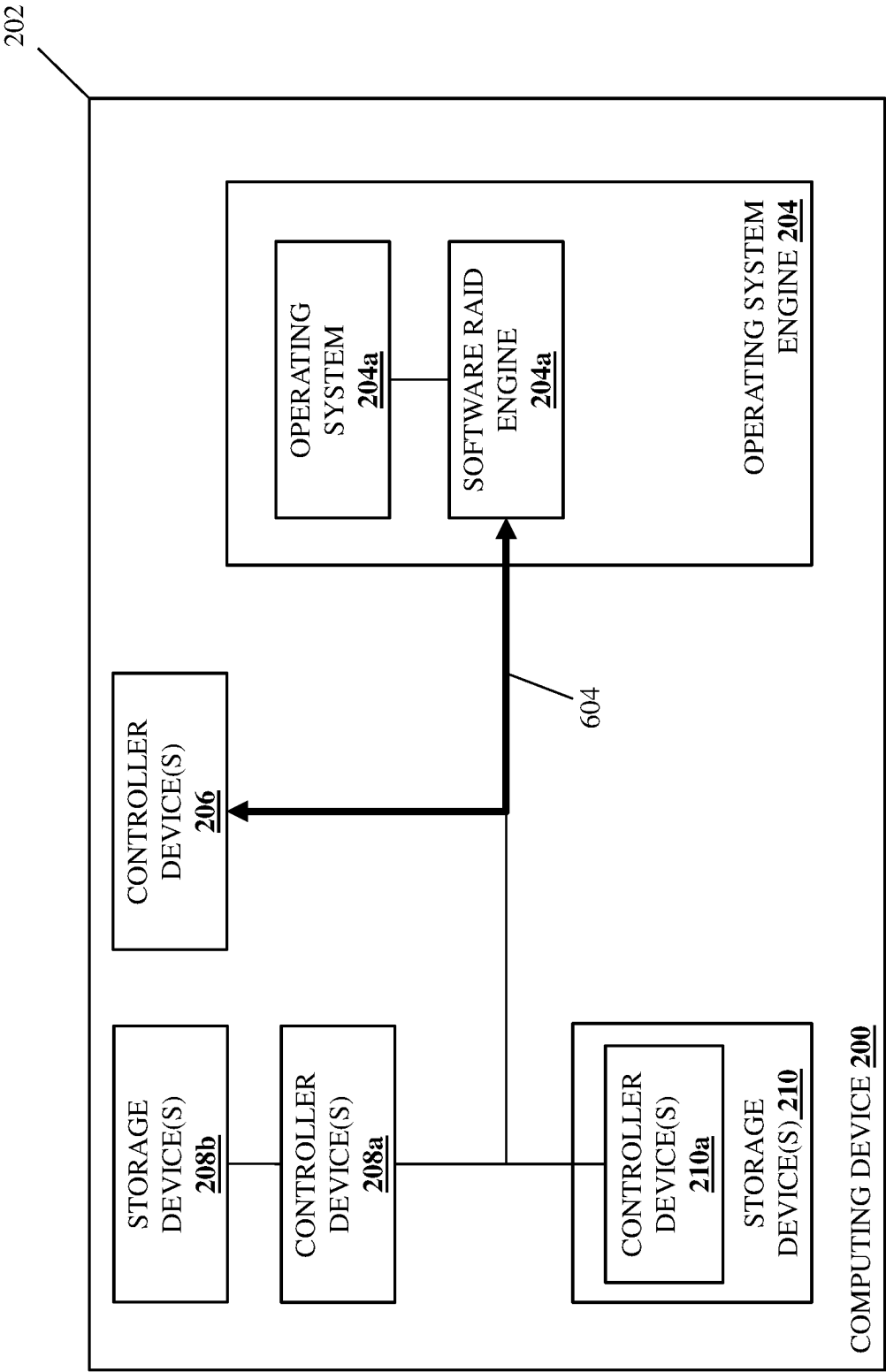


FIG. 6C

202

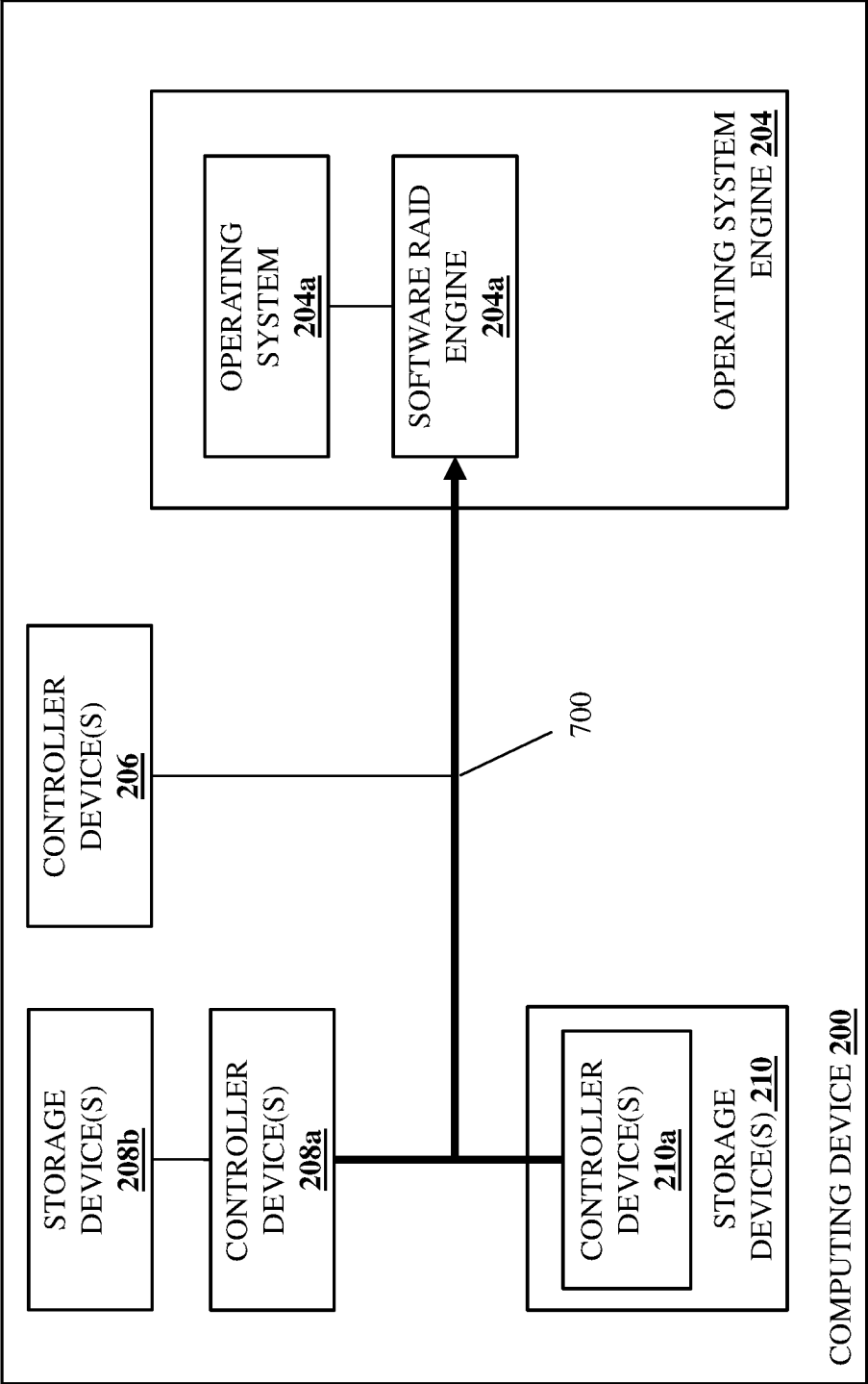


FIG. 7

202

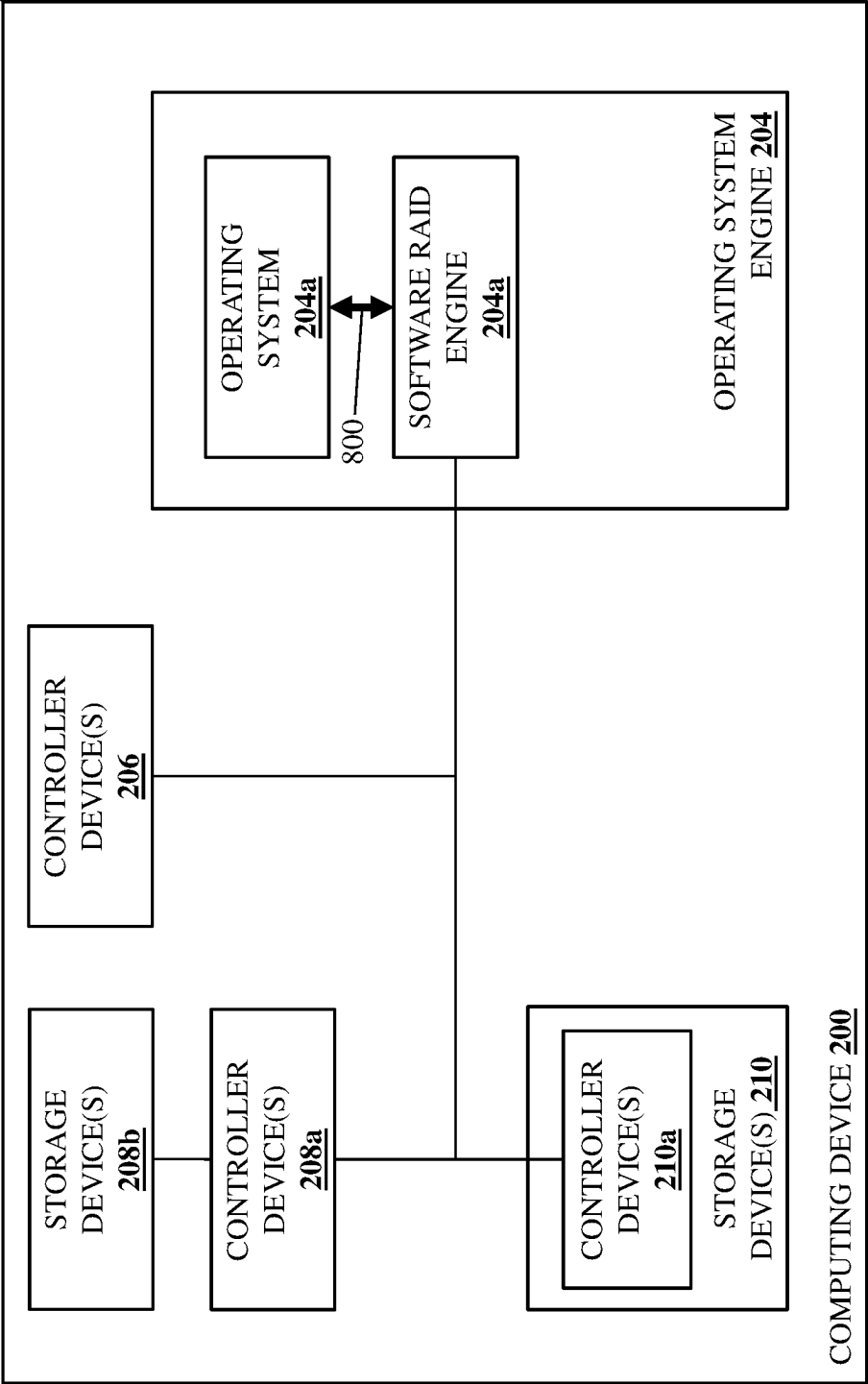


FIG. 8

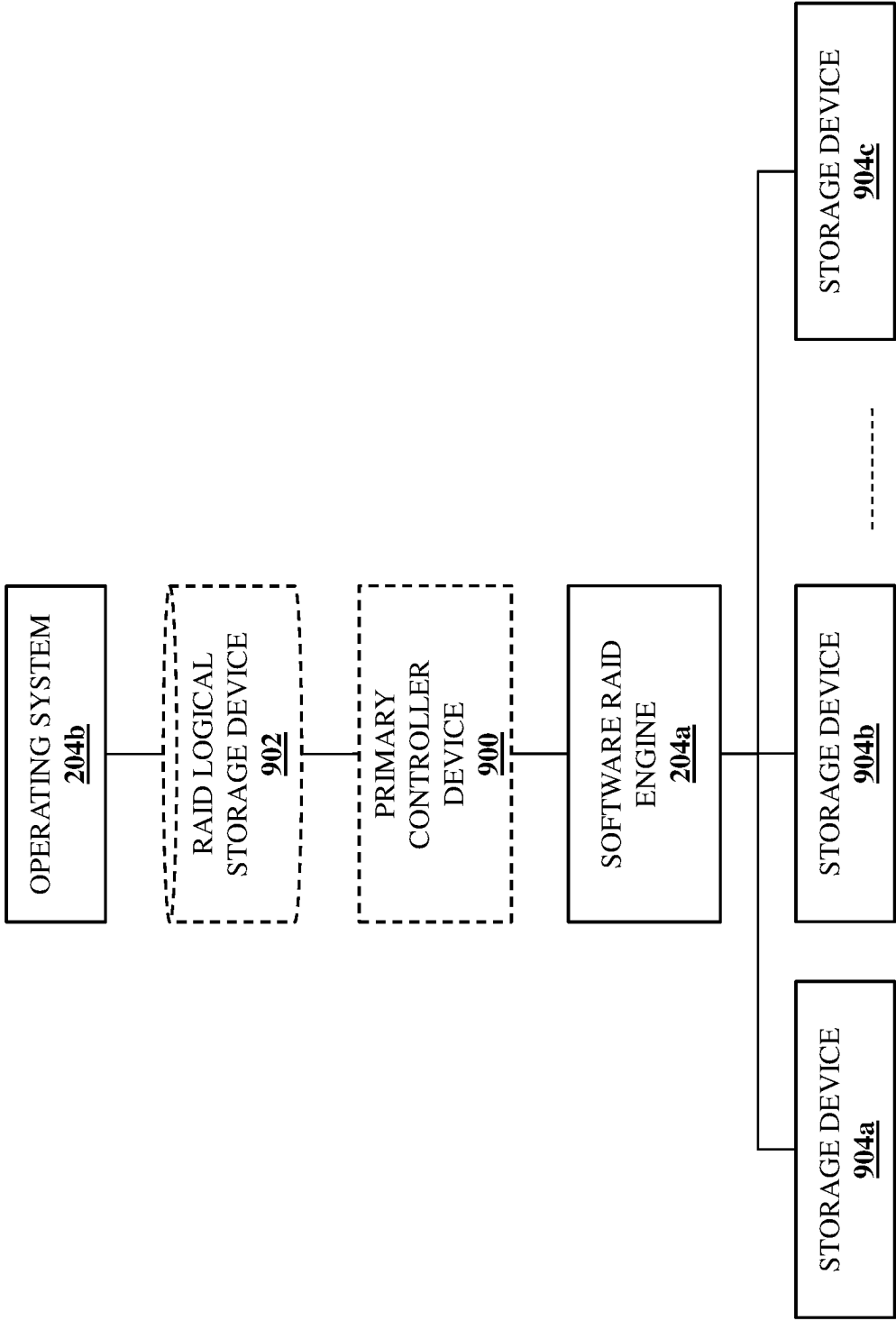


FIG. 9

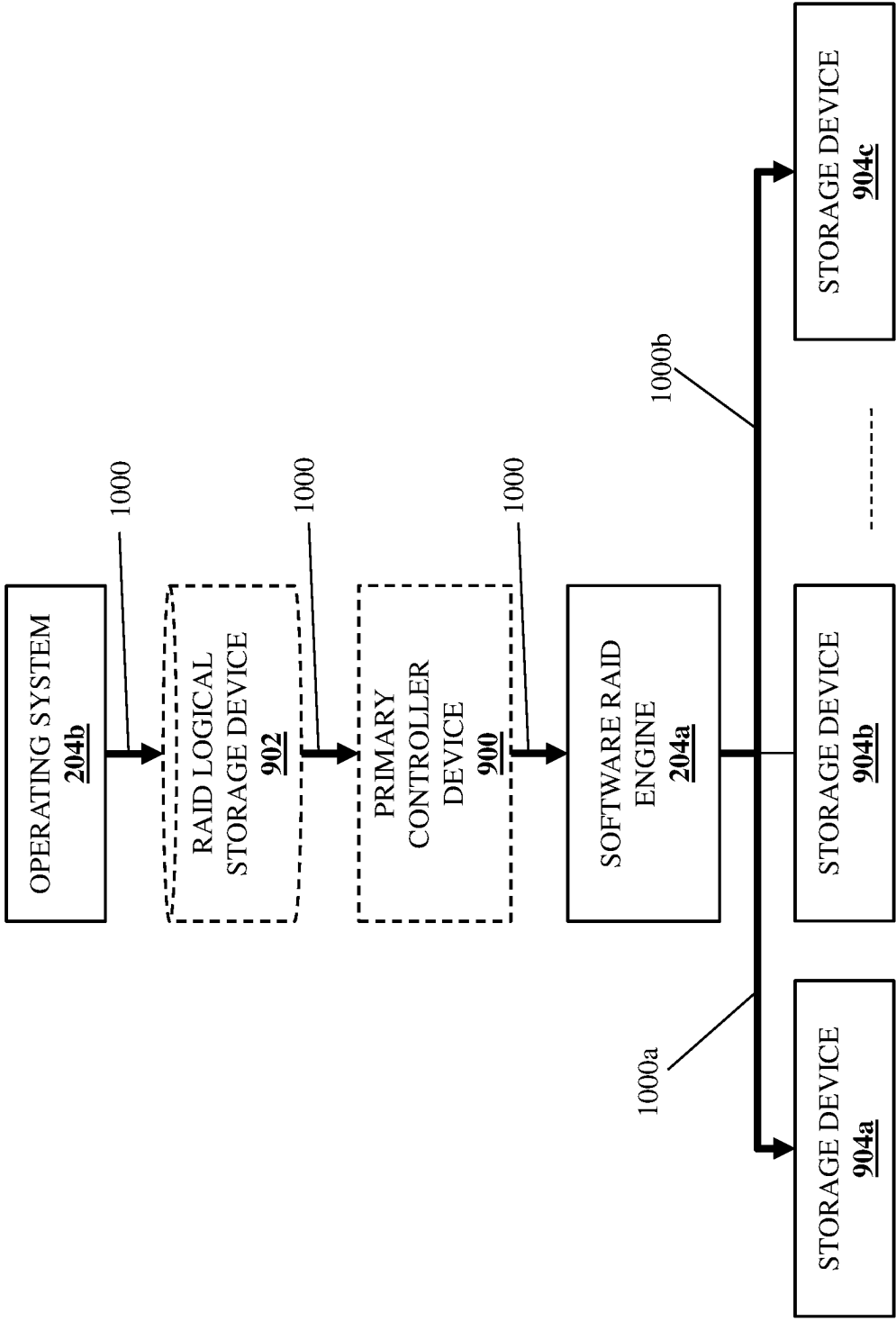


FIG. 10

1100

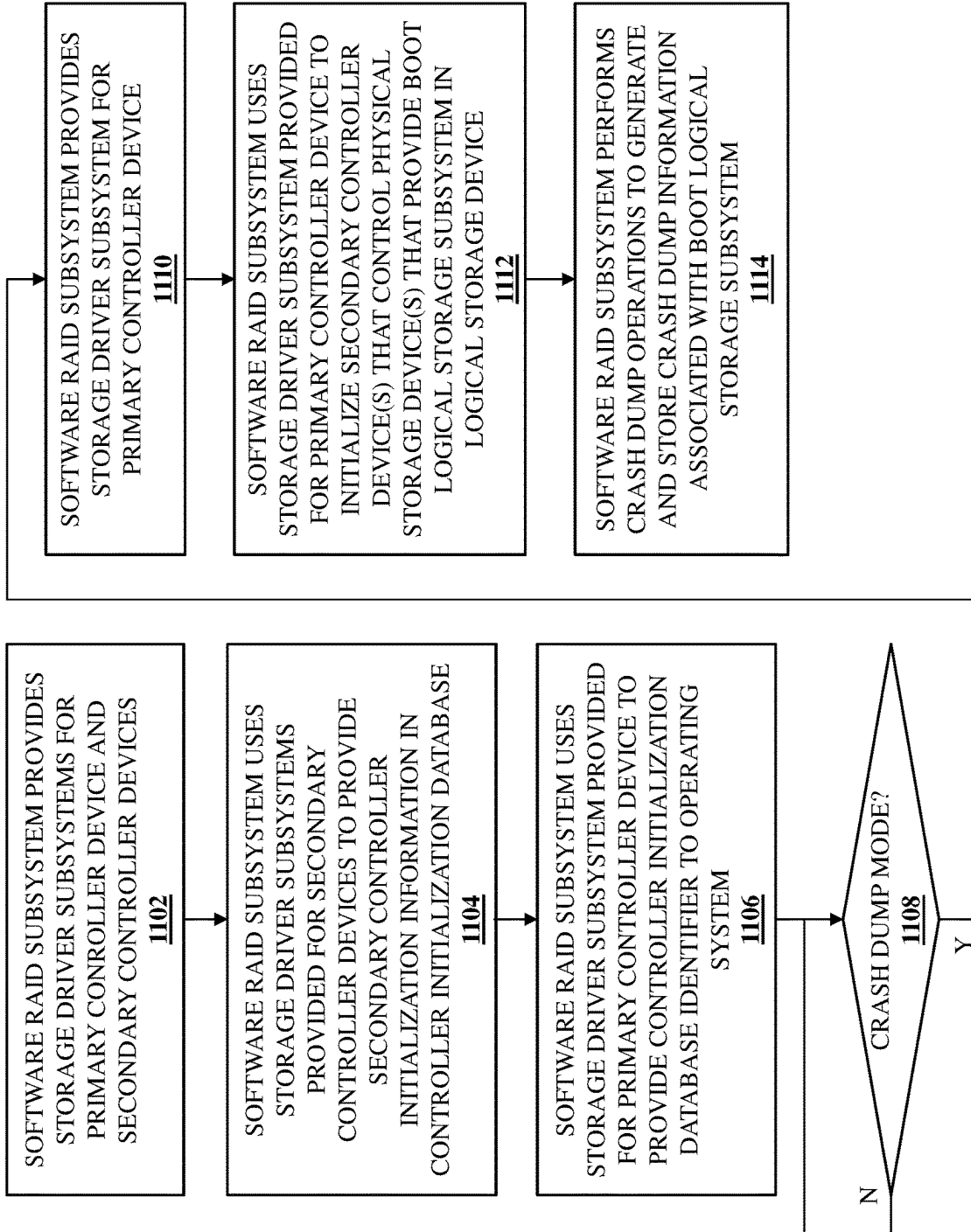


FIG 11

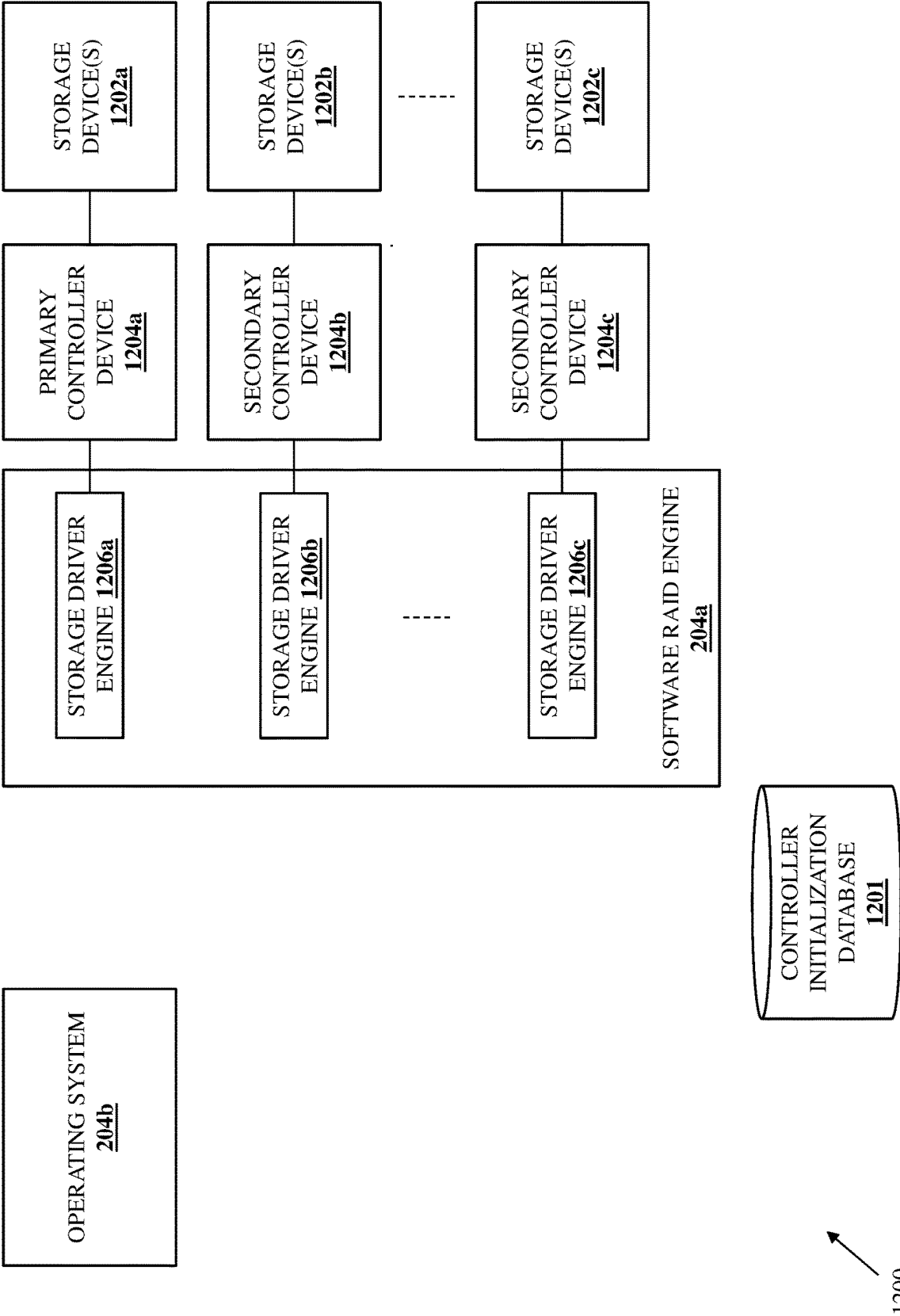
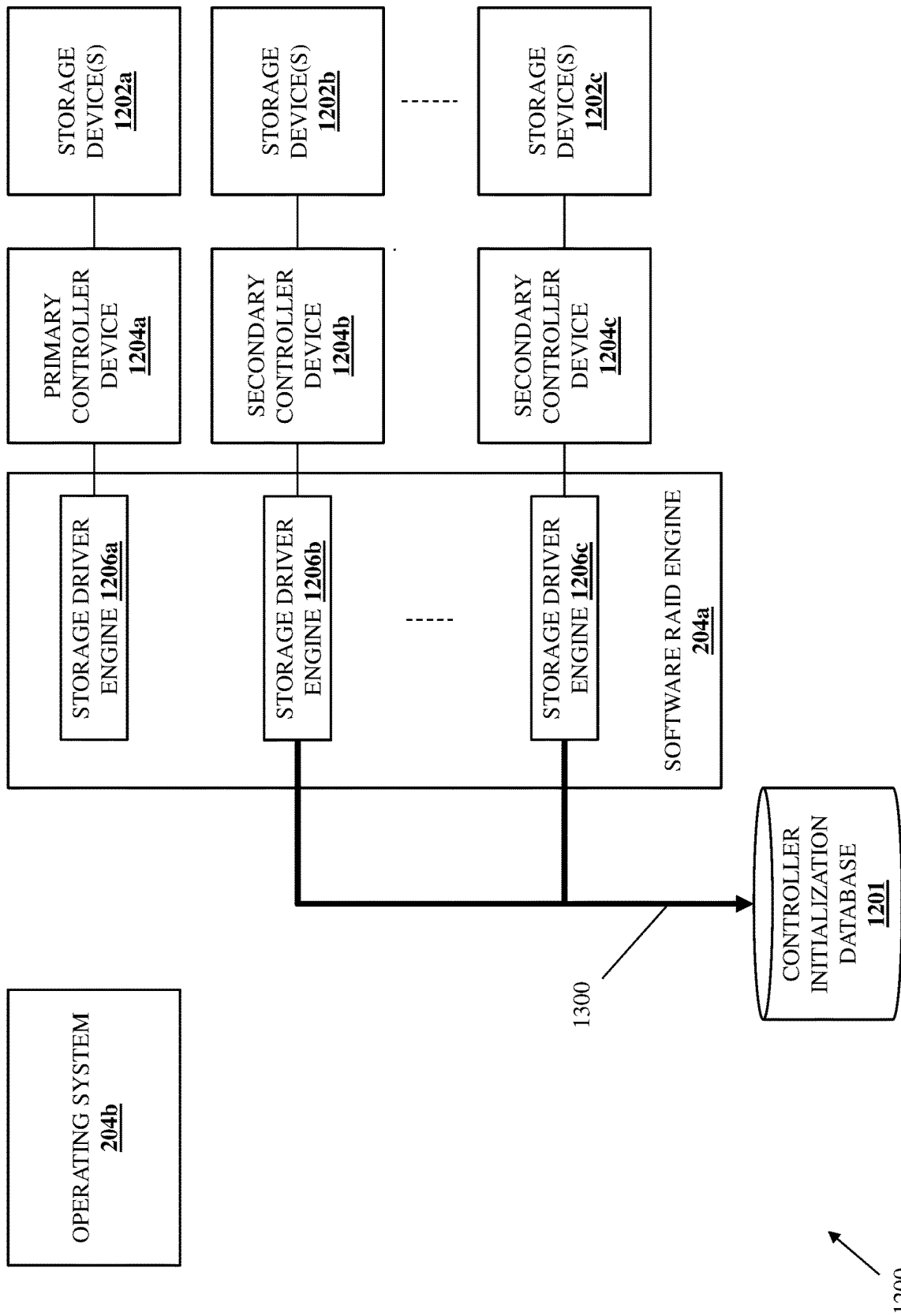


FIG. 12



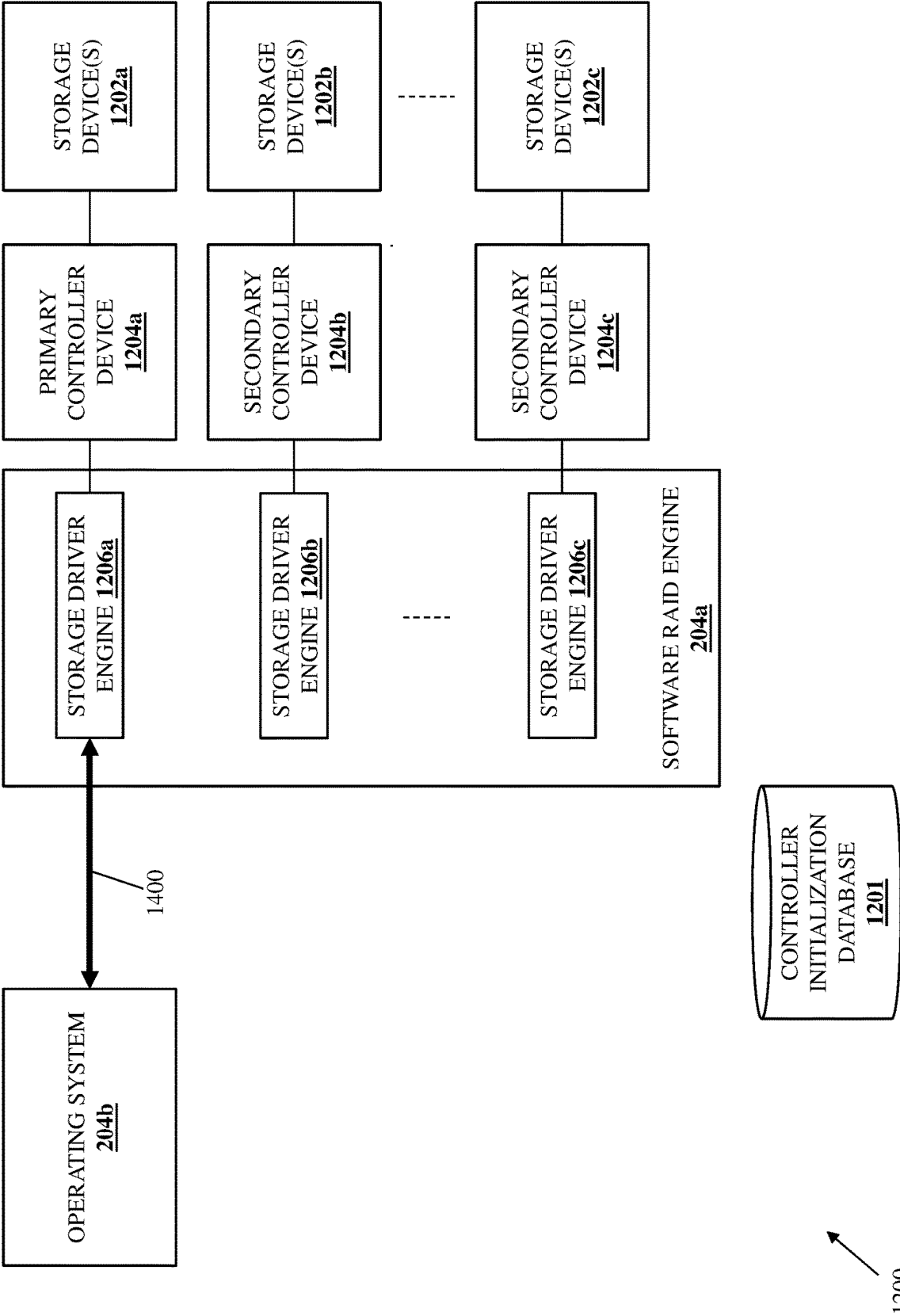


FIG. 14

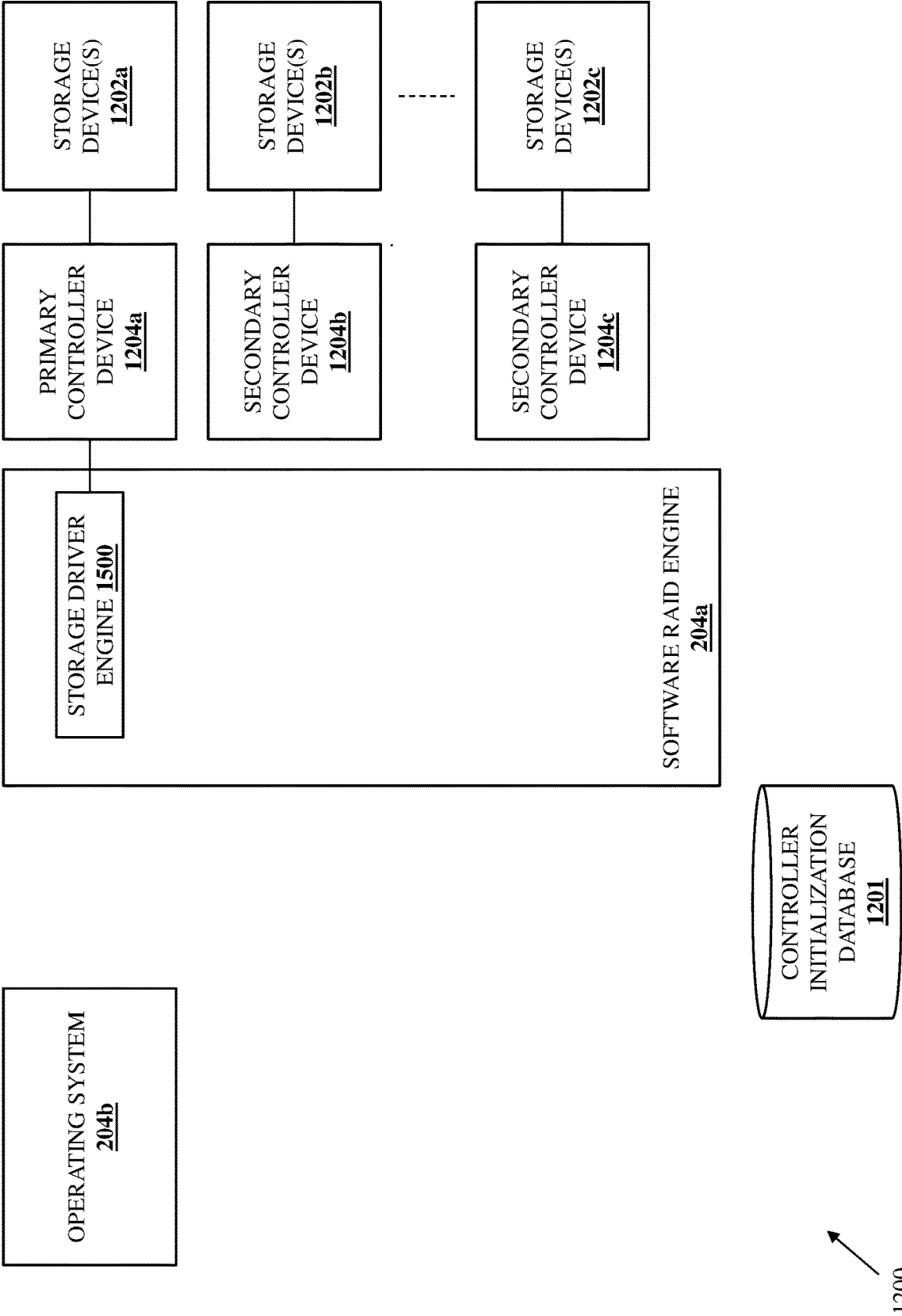


FIG. 15

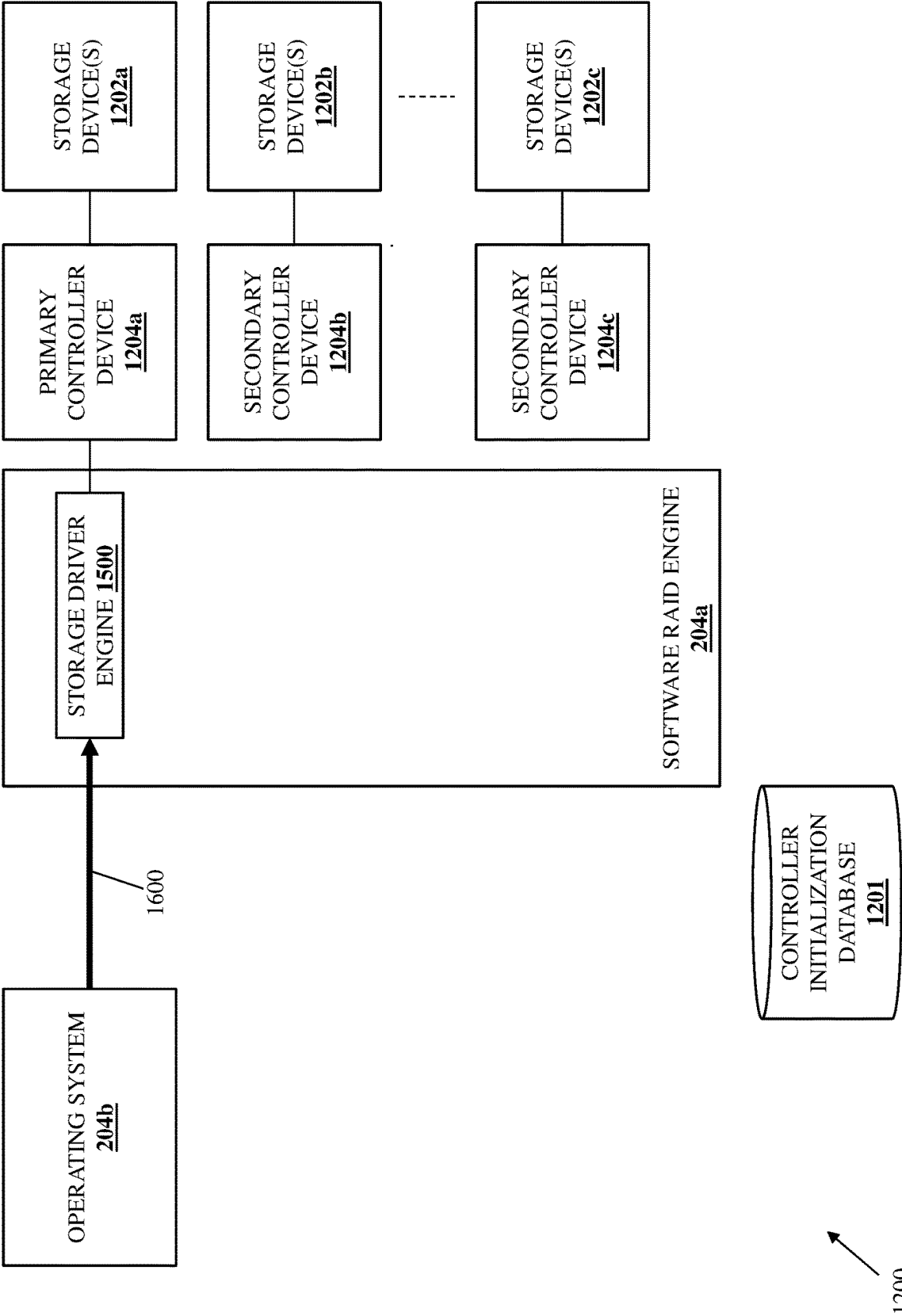


FIG. 16

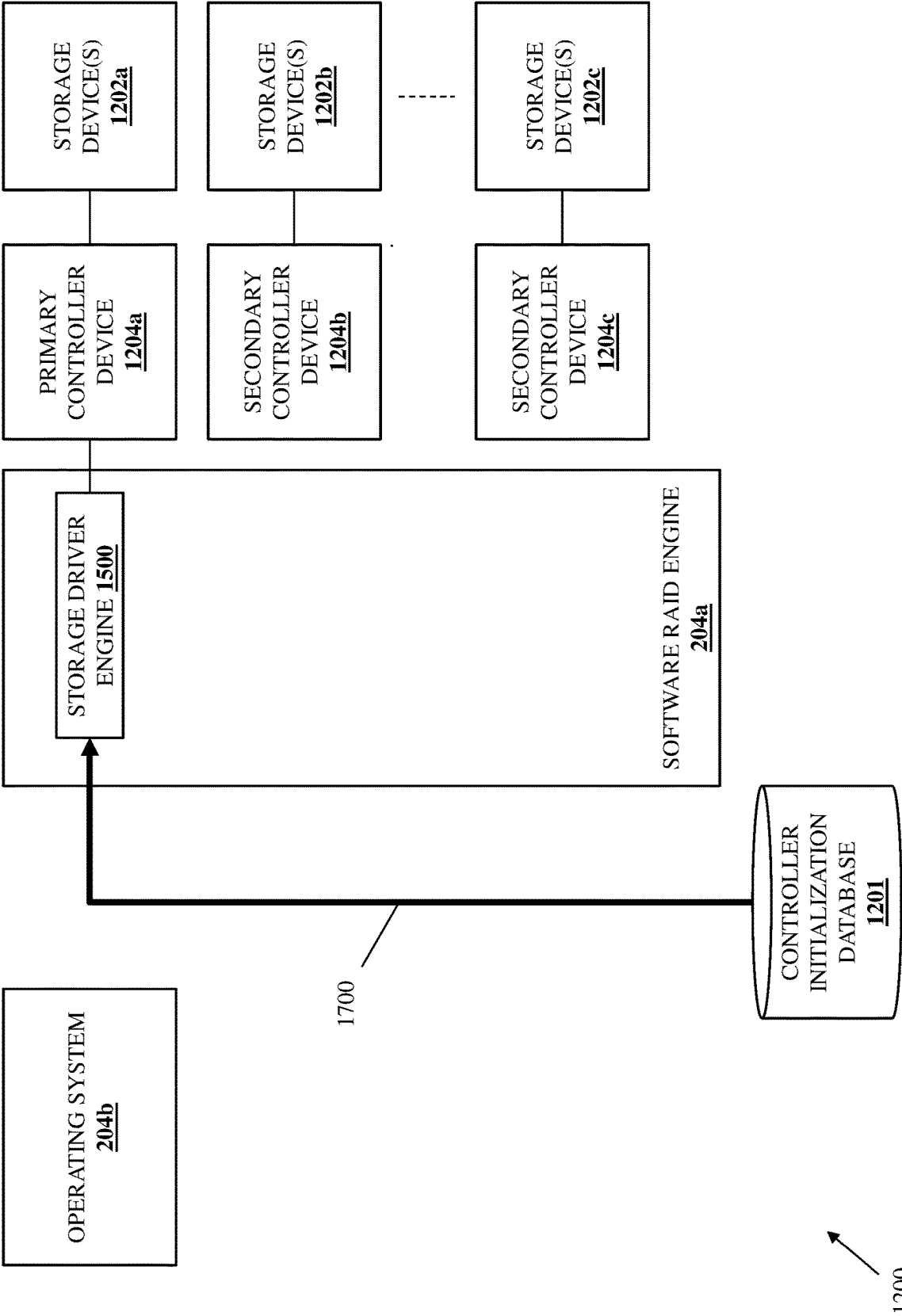
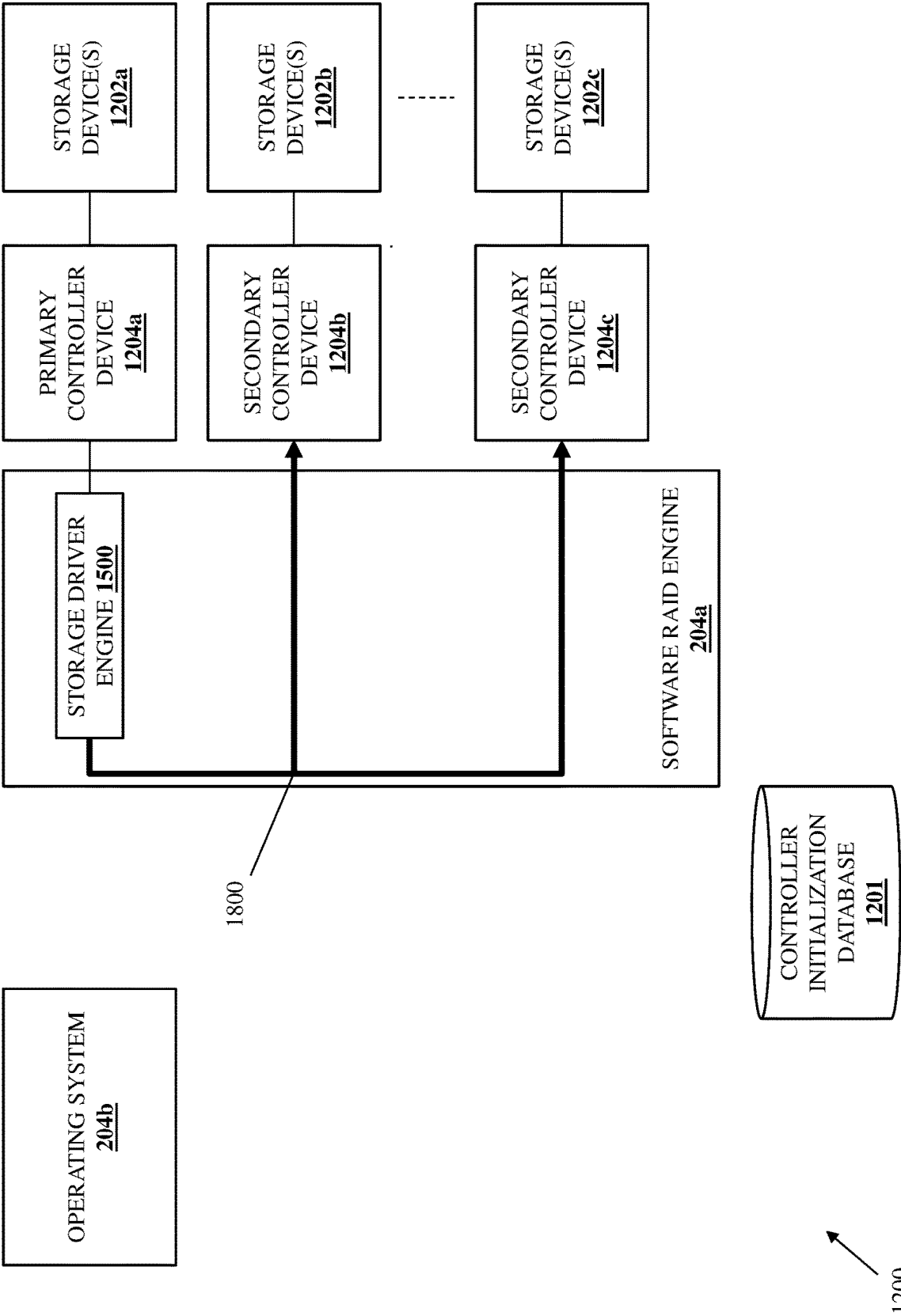
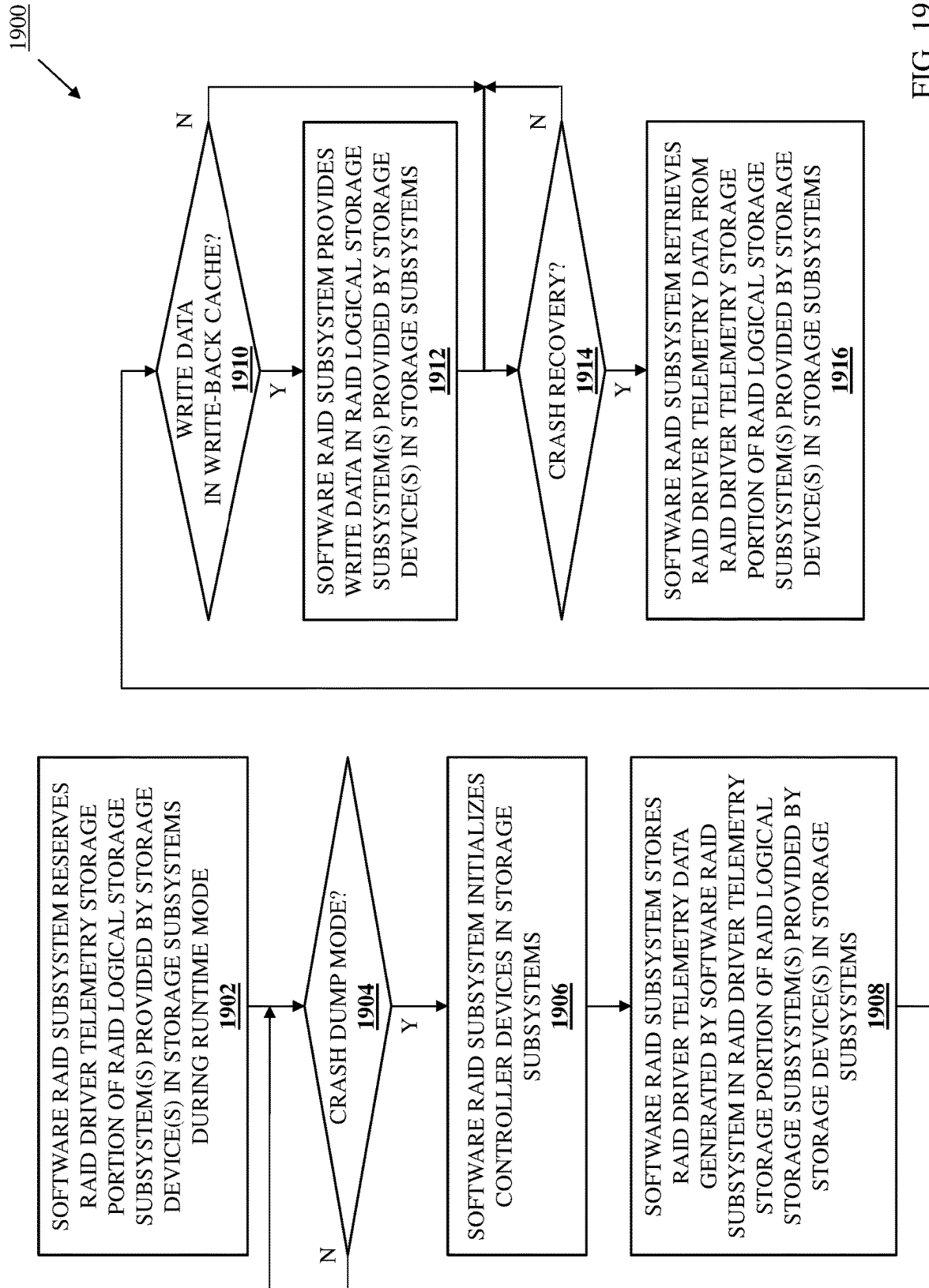


FIG. 17





2002

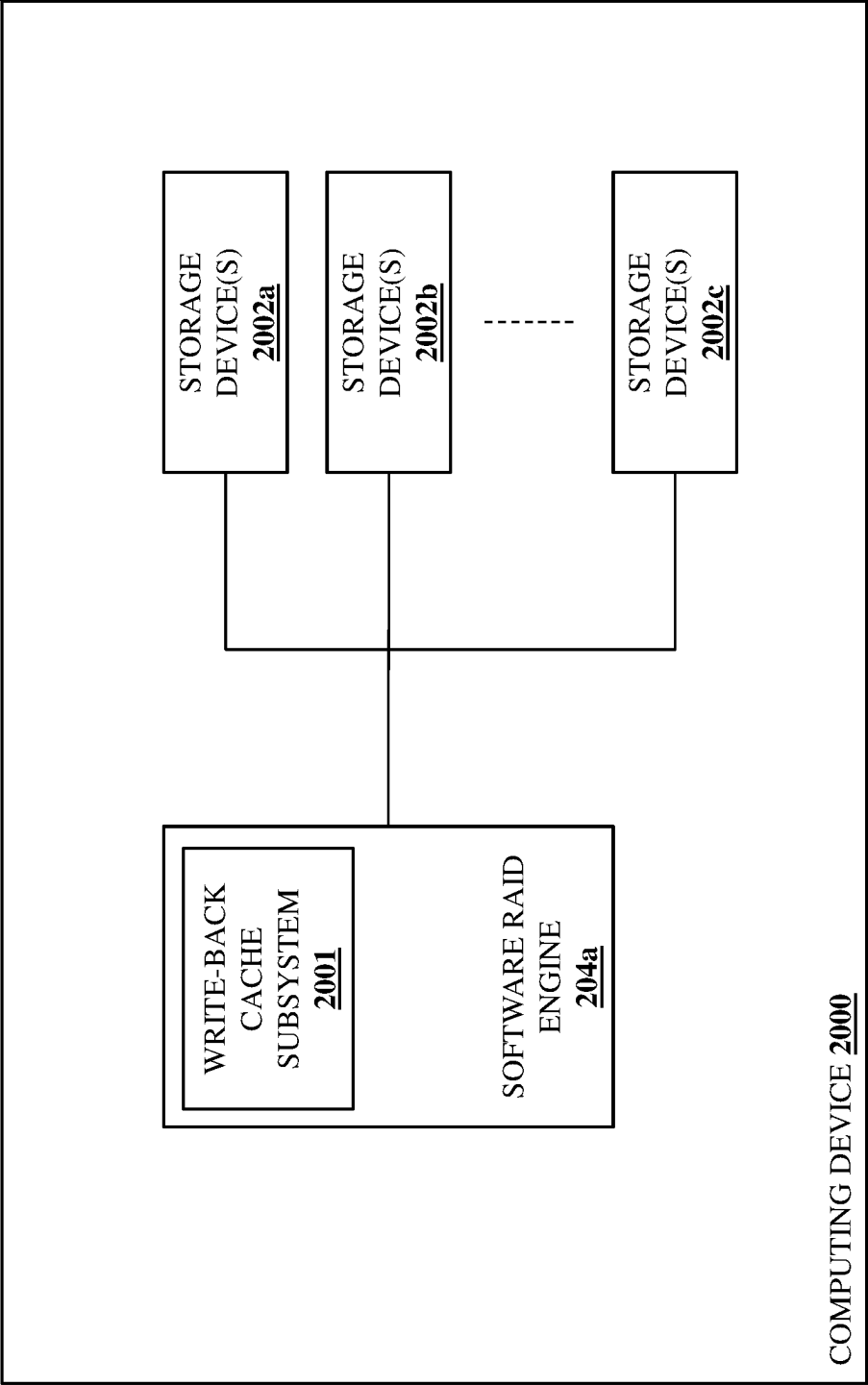


FIG. 20

2102

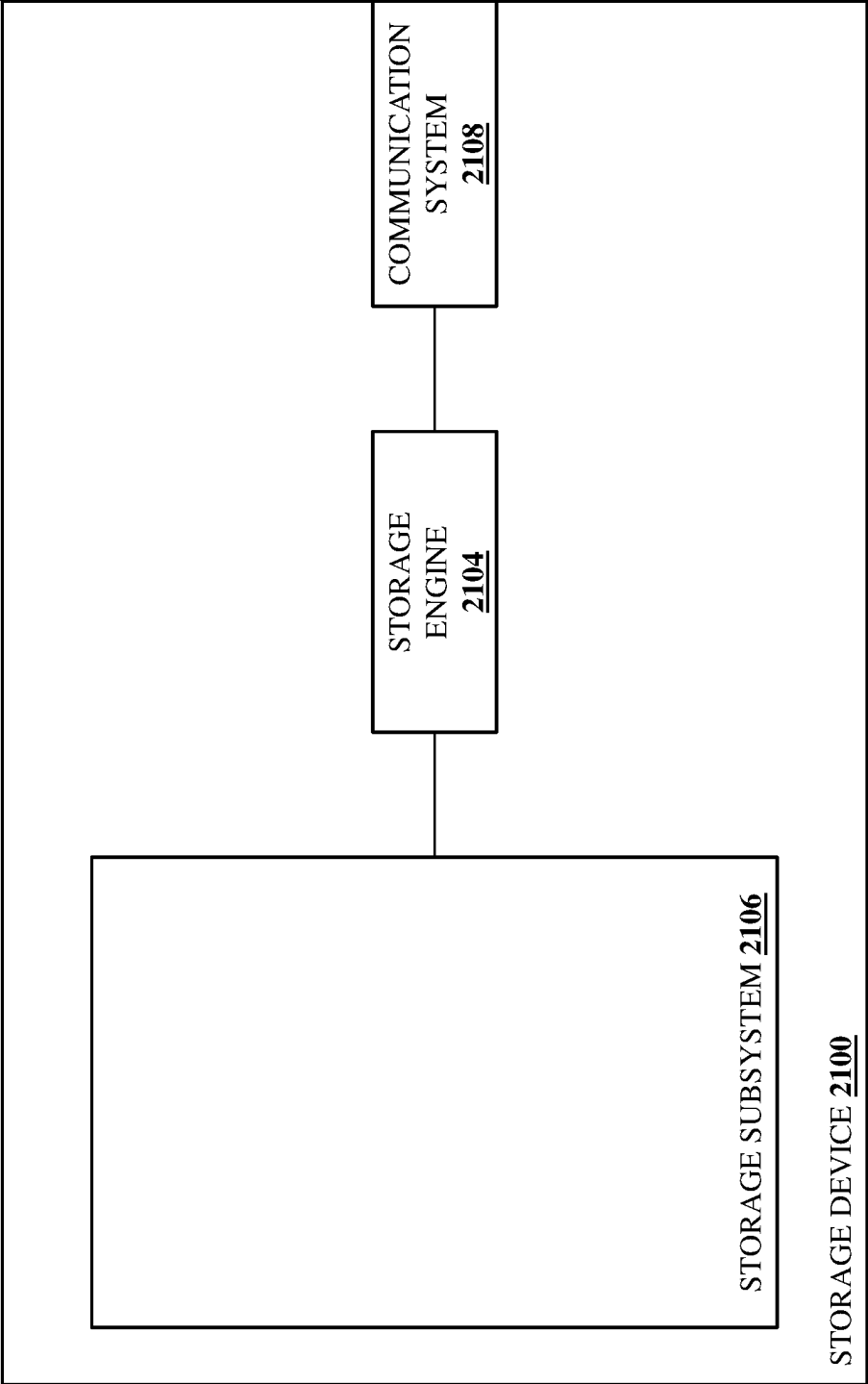


FIG. 21

2002

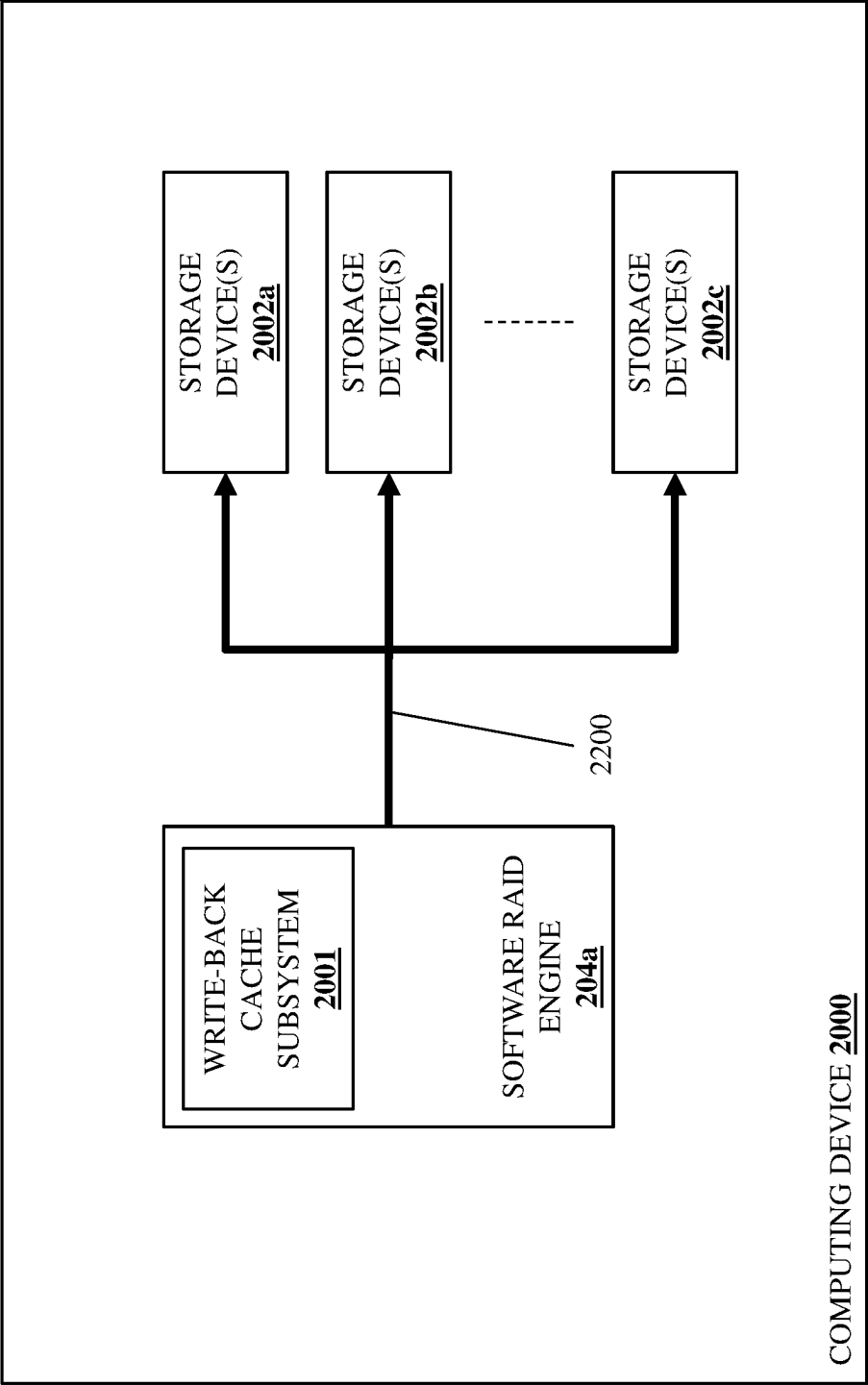


FIG. 22A

2102

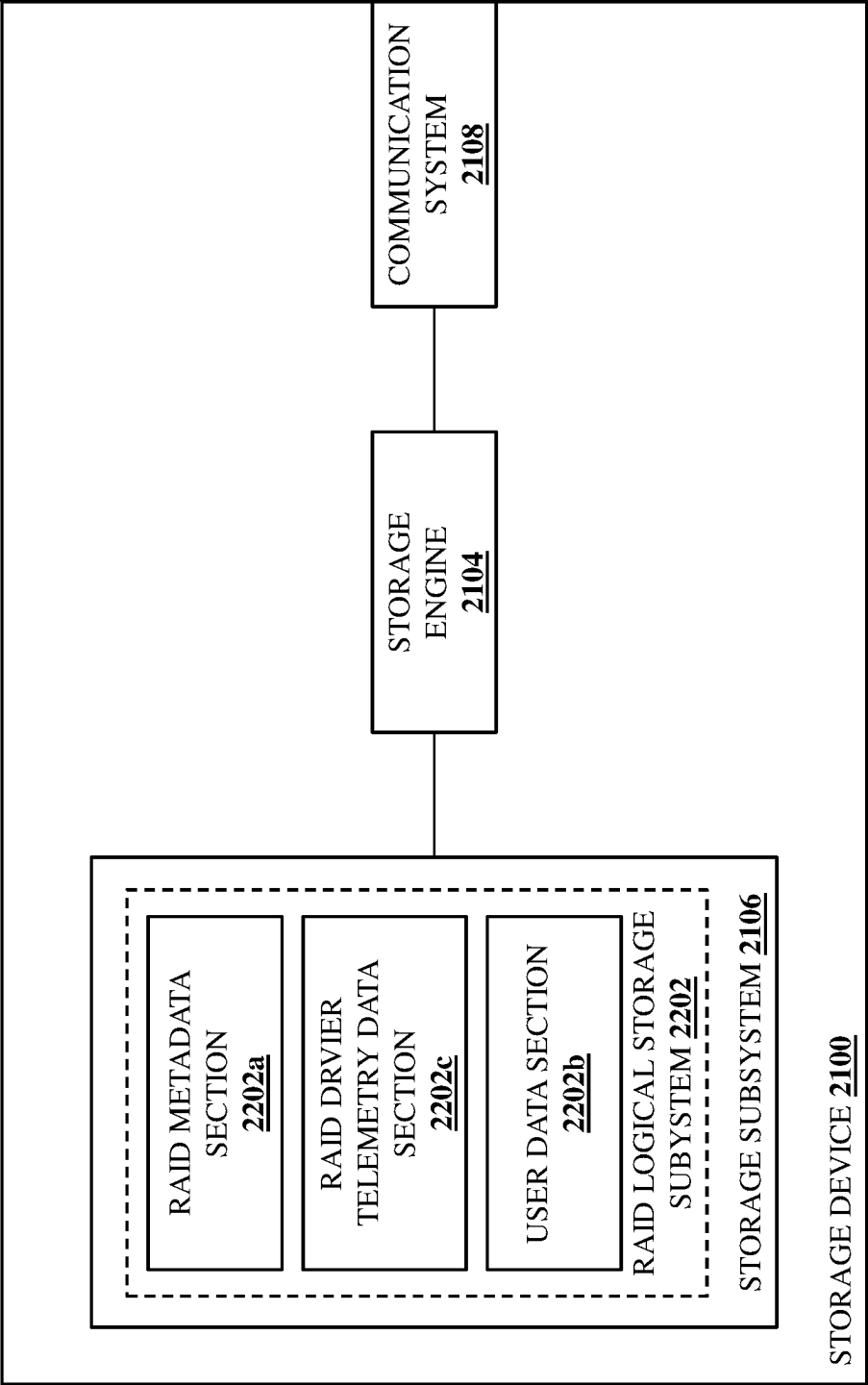


FIG. 22B

2002

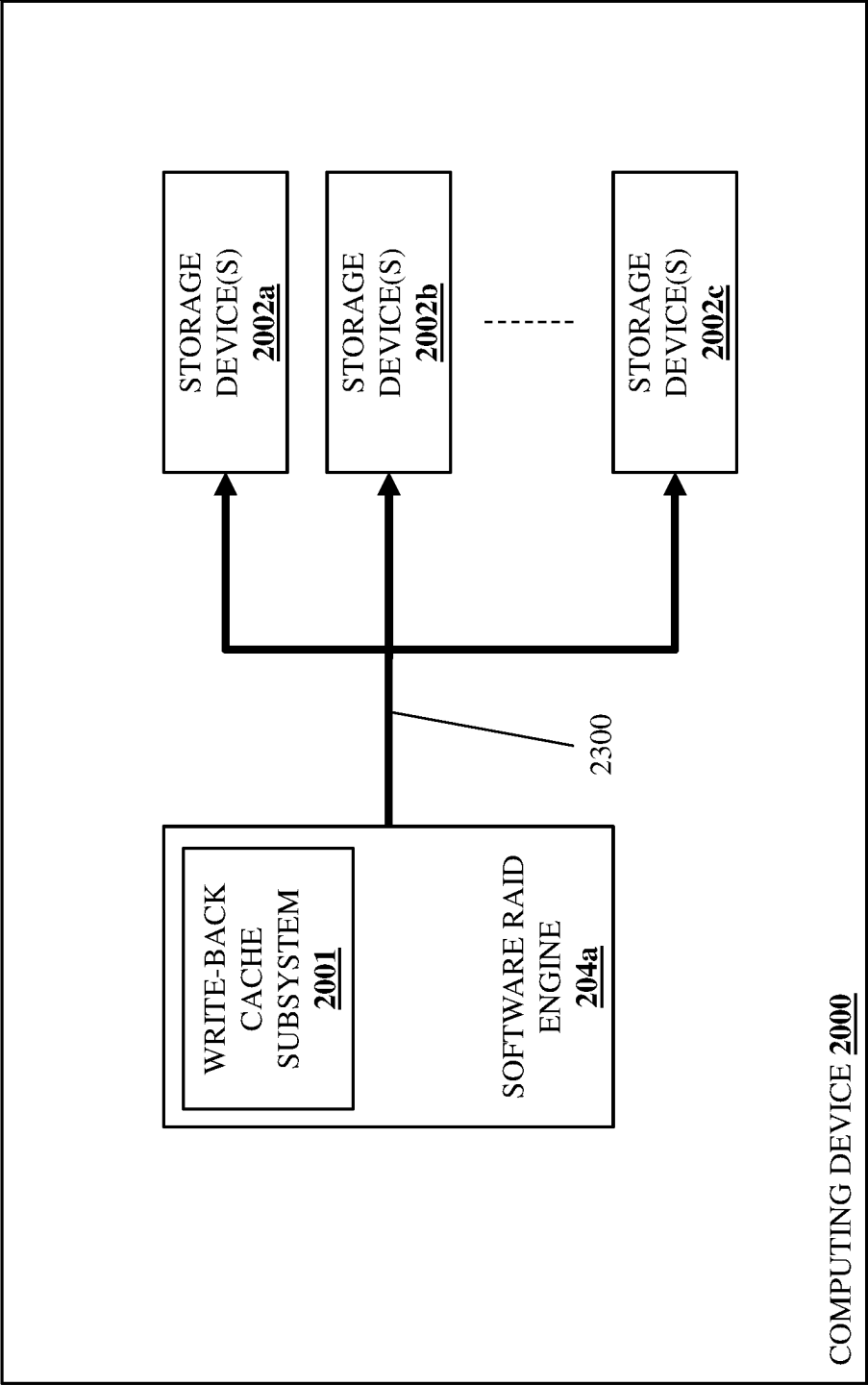


FIG. 23A

2102

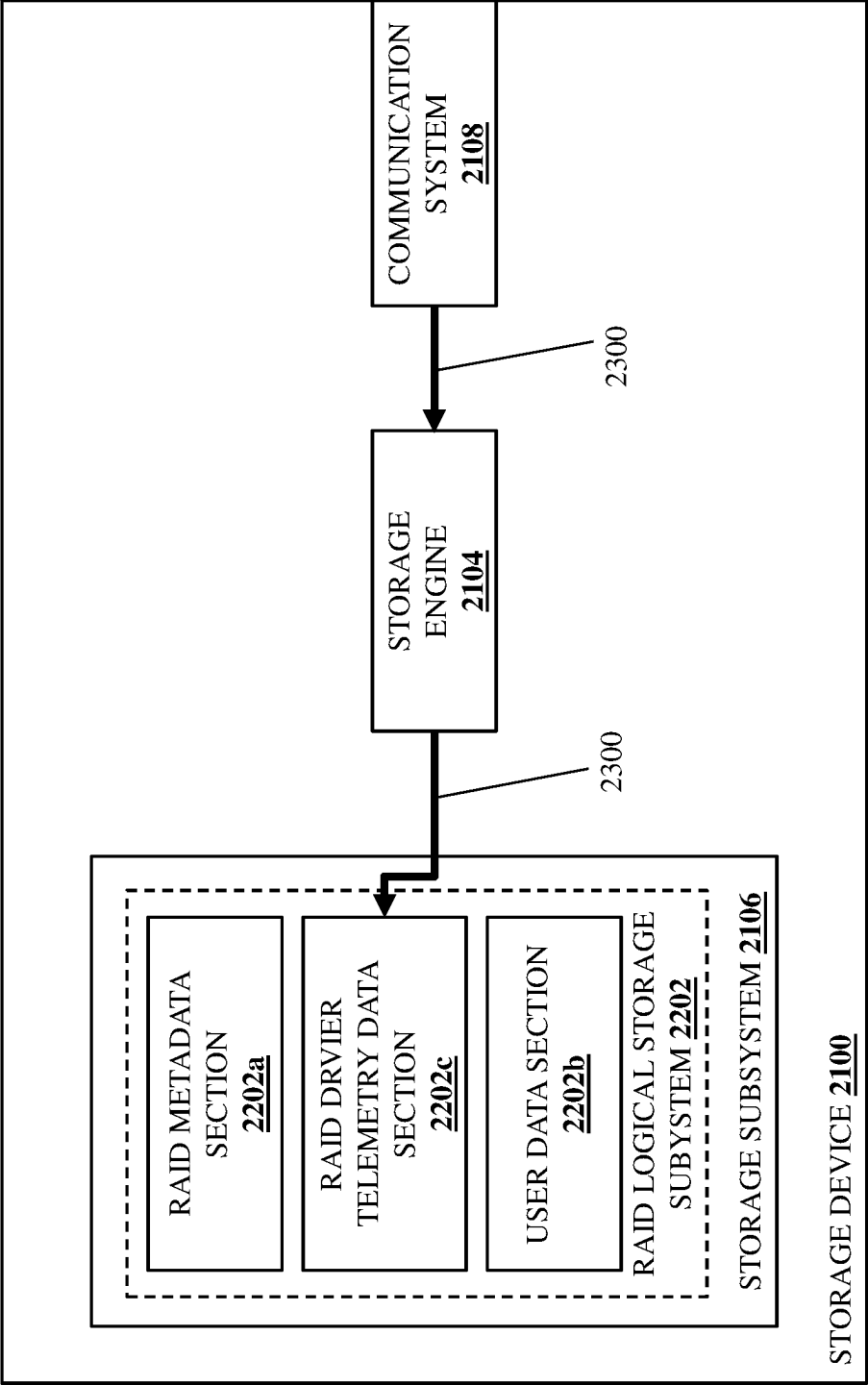


FIG. 23B

2002

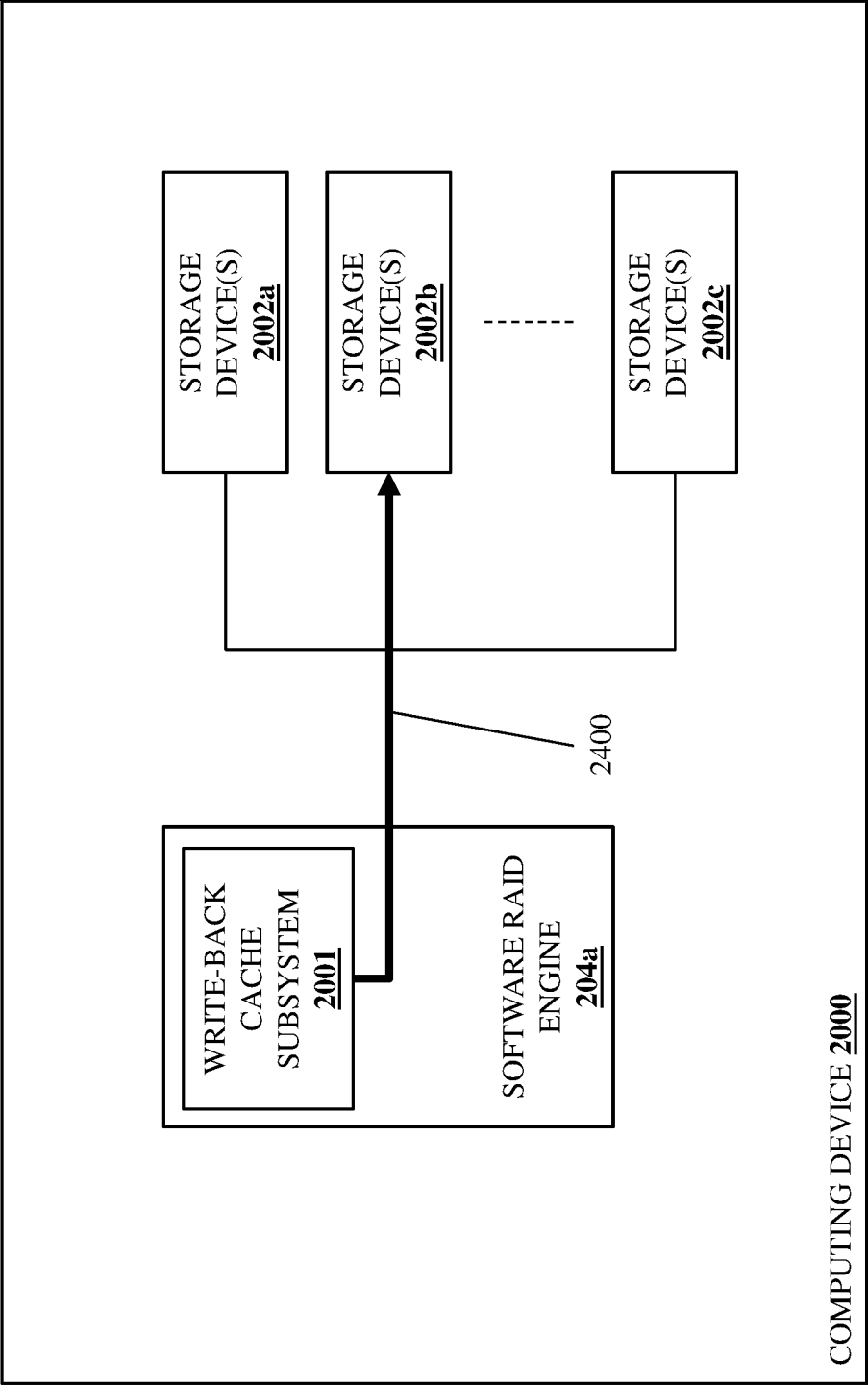


FIG. 24A

2102

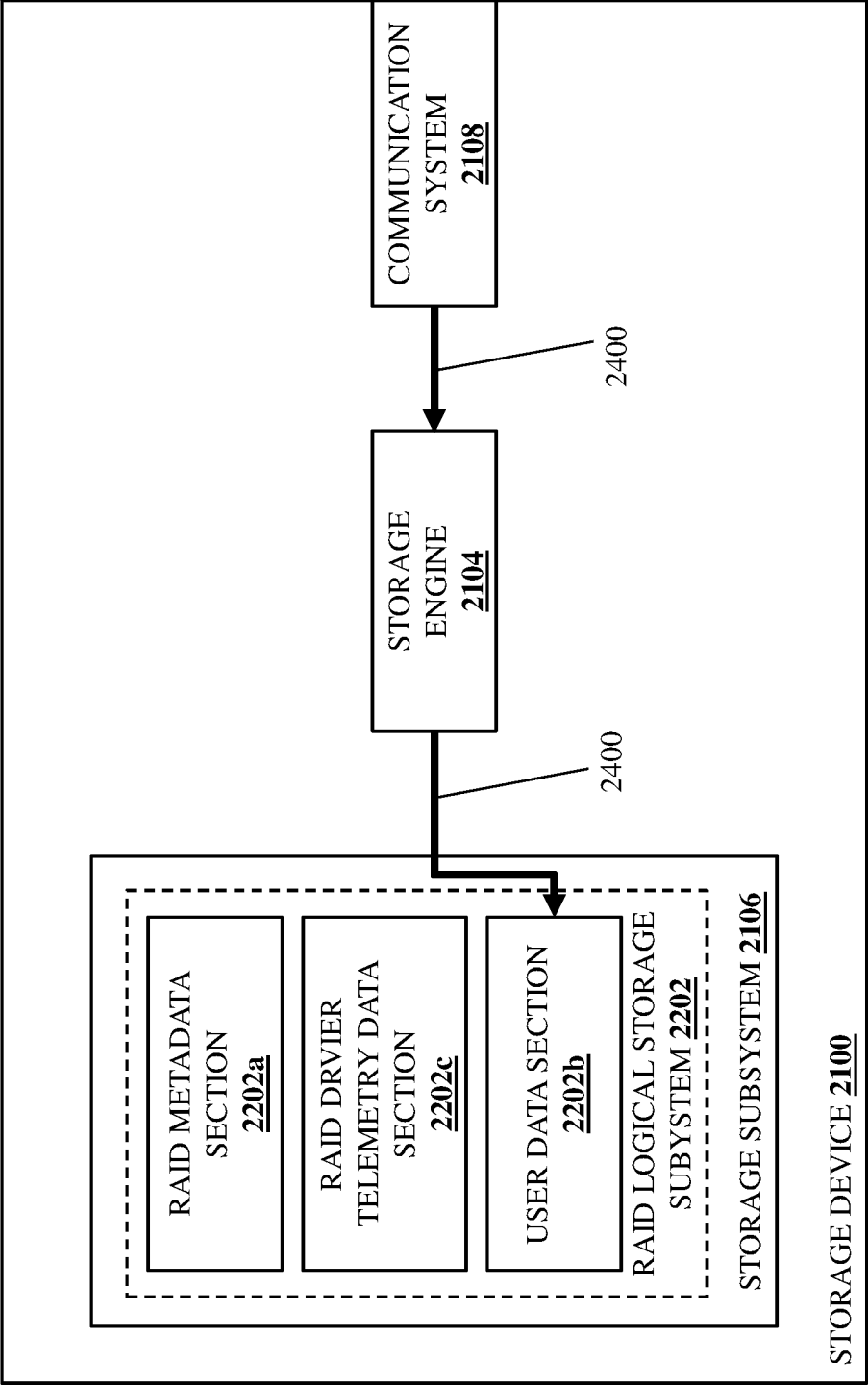


FIG. 24B

2102

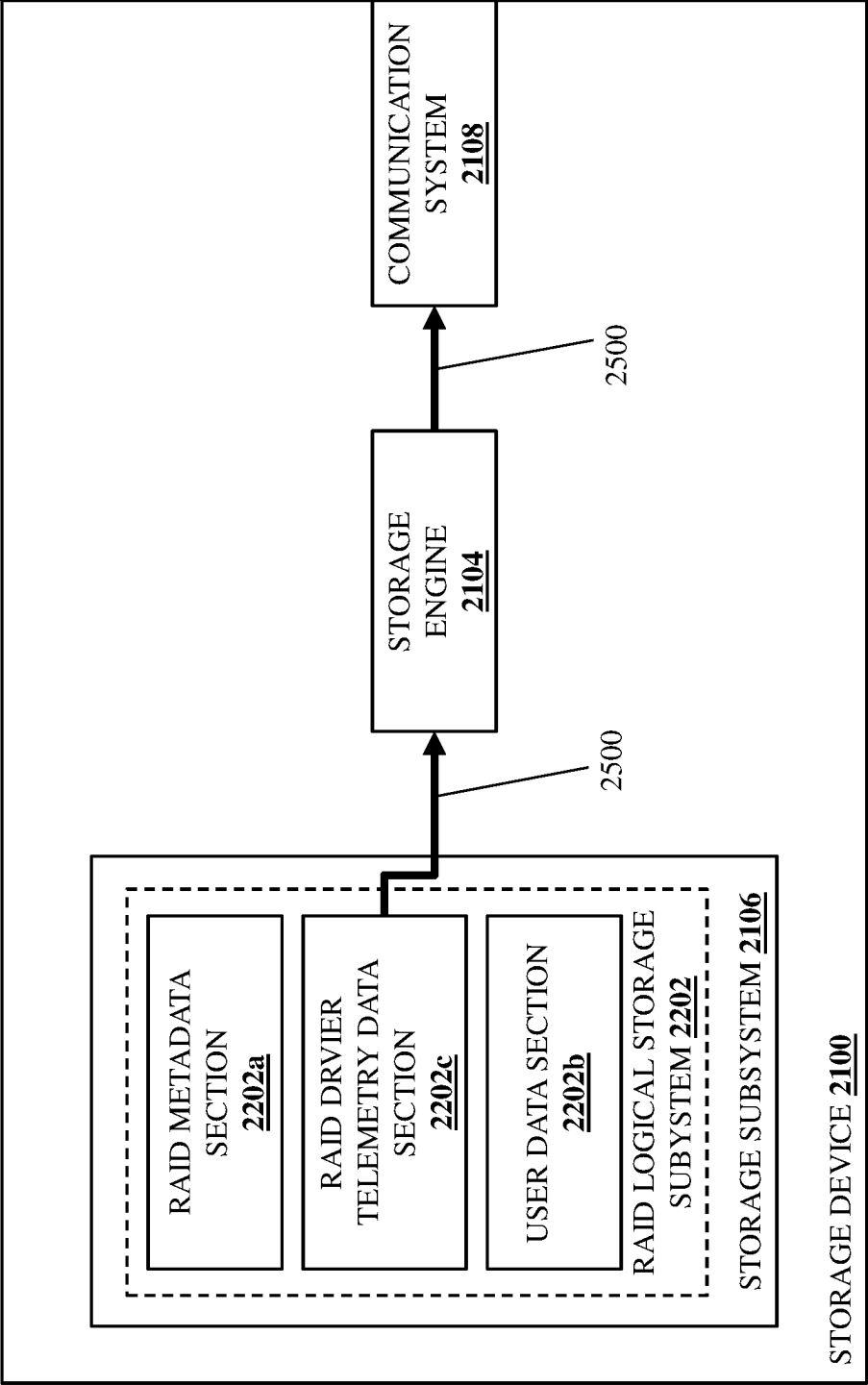


FIG. 25A

2002

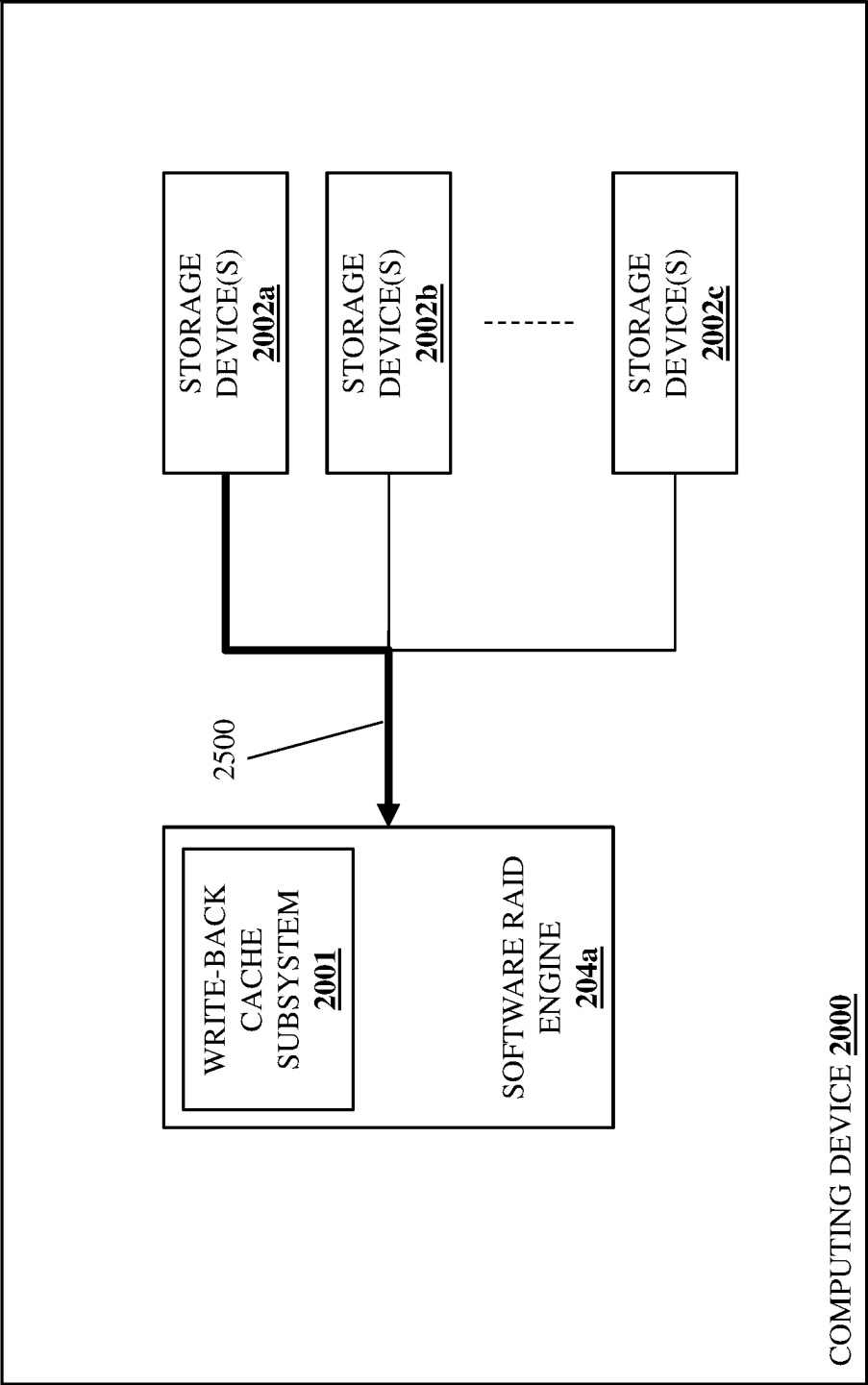


FIG. 25B

CRASH-BASED RAID DATA STORAGE SYSTEM

CROSS-REFERENCE TO RELATED APPLICATIONS

The present application is related to the following co-pending applications: (1) U.S. patent application Ser. No. 18/228,006, filed Jul. 31, 2023; (2) U.S. patent application Ser. No. 18/228,015, filed Jul. 31, 2023; (3) U.S. patent application Ser. No. 18/228,028, filed Jul. 31, 2023; and (4) U.S. patent application Ser. No. 18/228,055, filed Jul. 31, 2023.

BACKGROUND

The present disclosure relates generally to information handling systems, and more particularly to providing for the storage of Redundant Array of Independent Disk (RAID) data in response to a crash of an information handling system.

As the value and use of information continues to increase, individuals and businesses seek additional ways to process and store information. One option available to users is information handling systems. An information handling system generally processes, compiles, stores, and/or communicates information or data for business, personal, or other purposes thereby allowing users to take advantage of the value of the information. Because technology and information handling needs and requirements vary between different users or applications, information handling systems may also vary regarding what information is handled, how the information is handled, how much information is processed, stored, or communicated, and how quickly and efficiently the information may be processed, stored, or communicated. The variations in information handling systems allow for information handling systems to be general or configured for a specific user or specific use such as financial transaction processing, airline reservations, enterprise data storage, or global communications. In addition, information handling systems may include a variety of hardware and software components that may be configured to process, store, and communicate information and may include one or more computer systems, data storage systems, and networking systems.

Information handling systems such as, for example, operating systems in server devices and/or other computing devices known in the art use connected storage devices to store their data, and the protection of such data in the event of the unavailability of any of those storage devices is desirable. An option for protecting data stored on storage devices like those described above includes the utilization of Redundant Array of Independent Disk (RAID) techniques, which one of skill in the art in possession of the present disclosure will recognize is a data storage virtualization technology that combines multiple physical storage devices into one or more logical storage devices for the purposes of data redundancy, performance improvements, and/or other RAID benefits that would be apparent to one of skill in the art in possession of the present disclosure. However, the utilization of RAID techniques on data stored by an operating system in a server device can raise some issues.

For example, the WINDOWS® operating system available from MICROSOFT® Corp. of Redmond, Washington, United States, utilizes a “storage spaces” data protection solution that may be configured to protect data from storage device unavailability discussed above by aggregating stor-

age devices connected to a server/host device via the RAID techniques discussed above, and creating a logical storage device. However, such “storage spaces” data protection solutions do not provide the ability to configure the logical storage device during pre-boot or other initialization operations for the server device, which prevents the installation of the WINDOWS® operating system on that logical storage device. One solution to such issues is the provisioning of a hardware-based RAID data protection solution in the server device that implements hardware-based RAID data protection techniques on data stored in the storage devices in the server device, but such hardware-based RAID data protection solutions are relatively expensive.

Another solution to such issues is the use of a software RAID data protection solution in the server device like that provided in the Virtual RAID on Central Processing Unit (CPU) (VROC) data protection solution available in CPUs provided by INTEL® corporation of Santa Clara, California, United States. However, the VROC data protection solution discussed above is relatively limited in that it is only provided with INTEL® CPUs, may only be utilized with Non-Volatile Memory express (NVMe) storage devices and Serial AT Attachment (SATA) storage devices, only provides RAID1 and RAID5 data protection, and requires Volume Management Device (VMD) hardware in the CPU. As such, the VROC data protection solutions discussed above do not operate with some types of storage devices (e.g., Serial Attached Small Computer System Interface (SCSI) (SAS) storage devices) or some types of CPUs (e.g., CPUs available from Advance Micro Devices (AMD) of Santa Clara, California, United States). Furthermore, the VROC data protection solutions discussed above are also associated with relatively high licensing costs.

Accordingly, it would be desirable to provide a direct-attached storage device software RAID system that addresses the issues discussed above.

SUMMARY

According to one embodiment, an Information Handling System (IHS) includes a processing system; and a memory system that is coupled to the processing system and that includes instructions that, when executed by the processing system, cause the processing system to provide a software Redundant Array of Independent Disk (RAID) engine that is configured to: reserve, during a runtime mode of an operating system that is coupled to the processing system, a RAID driver telemetry storage portion of a RAID logical storage subsystem that is provided by at least one storage device in a plurality of storage subsystems that are coupled to the processing system; initialize, in response to the operating system entering a crash dump mode from the runtime mode, respective controller devices in each of the plurality of storage subsystems; and store, in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the operating system, RAID driver telemetry data generated by the software RAID subsystem.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a schematic view illustrating an embodiment of an Information Handling System (IHS).

3

FIG. 2 is a schematic view illustrating an embodiment of a computing device that may include the direct-attached storage device software RAID system of the present disclosure.

FIG. 3 is a flow chart illustrating an embodiment of a method for providing a software RAID using direct-attached storage devices in a computing device.

FIG. 4 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 5 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 6A is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 6B is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 6C is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 7 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 8 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 9 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 10 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 3.

FIG. 11 is a flow chart illustrating an embodiment of a method for providing crash dumps in a software RAID using direct-attached storage devices in a computing device.

FIG. 12 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 2 during the method of FIG. 11.

FIG. 13 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 14 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 15 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 16 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 17 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 18 is a schematic view illustrating an embodiment of the operation of the computing device of FIG. 12 during the method of FIG. 11.

FIG. 19 is a flow chart illustrating an embodiment of a method for storing RAID data in response to a crash of a computing device.

FIG. 20 is a schematic view illustrating an embodiment of the computing device of FIG. 2.

FIG. 21 is a schematic view illustrating an embodiment of a storage device that may be provided in the computing device of FIG. 20.

4

FIG. 22A is a schematic view illustrating an embodiment of the computing device of FIG. 20 operating during the method of FIG. 21.

FIG. 22B is a schematic view illustrating an embodiment of the storage device of FIG. 21 operating during the method of FIG. 21.

FIG. 23A is a schematic view illustrating an embodiment of the computing device of FIG. 20 operating during the method of FIG. 21.

FIG. 23B is a schematic view illustrating an embodiment of the storage device of FIG. 21 operating during the method of FIG. 21.

FIG. 24A is a schematic view illustrating an embodiment of the computing device of FIG. 20 operating during the method of FIG. 21.

FIG. 24B is a schematic view illustrating an embodiment of the storage device of FIG. 21 operating during the method of FIG. 21.

FIG. 25A is a schematic view illustrating an embodiment of the storage device of FIG. 21 operating during the method of FIG. 21.

FIG. 25B is a schematic view illustrating an embodiment of the computing device of FIG. 20 operating during the method of FIG. 21.

DETAILED DESCRIPTION

For purposes of this disclosure, an information handling system may include any instrumentality or aggregate of instrumentalities operable to compute, calculate, determine, classify, process, transmit, receive, retrieve, originate, switch, store, display, communicate, manifest, detect, record, reproduce, handle, or utilize any form of information, intelligence, or data for business, scientific, control, or other purposes. For example, an information handling system may be a personal computer (e.g., desktop or laptop), tablet computer, mobile device (e.g., personal digital assistant (PDA) or smart phone), server (e.g., blade server or rack server), a network storage device, or any other suitable device and may vary in size, shape, performance, functionality, and price. The information handling system may include random access memory (RAM), one or more processing resources such as a central processing unit (CPU) or hardware or software control logic, ROM, and/or other types of nonvolatile memory. Additional components of the information handling system may include one or more disk drives, one or more network ports for communicating with external devices as well as various input and output (I/O) devices, such as a keyboard, a mouse, touchscreen and/or a video display. The information handling system may also include one or more buses operable to transmit communications between the various hardware components.

In one embodiment, IHS 100, FIG. 1, includes a processor 102, which is connected to a bus 104. Bus 104 serves as a connection between processor 102 and other components of IHS 100. An input device 106 is coupled to processor 102 to provide input to processor 102. Examples of input devices may include keyboards, touchscreens, pointing devices such as mice, trackballs, and trackpads, and/or a variety of other input devices known in the art. Programs and data are stored on a mass storage device 108, which is coupled to processor 102. Examples of mass storage devices may include hard discs, optical discs, magneto-optical discs, solid-state storage devices, and/or a variety of other mass storage devices known in the art. IHS 100 further includes a display 110, which is coupled to processor 102 by a video controller 112. A system memory 114 is coupled to processor

5

102 to provide the processor with fast storage to facilitate execution of computer programs by processor 102. Examples of system memory may include random access memory (RAM) devices such as dynamic RAM (DRAM), synchronous DRAM (SDRAM), solid state memory devices, and/or a variety of other memory devices known in the art. In an embodiment, a chassis 116 houses some or all of the components of IHS 100. It should be understood that other buses and intermediate circuits can be deployed between the components described above and processor 102 to facilitate interconnection between the components and the processor 102.

Referring now to FIG. 2, an embodiment of a computing device 200 is illustrated that may include the direct-attached storage device software RAID system of the present disclosure. In an embodiment, the computing device 200 may be provided by the IHS 100 discussed above with reference to FIG. 1 and/or may include some or all of the components of the IHS 100, and in specific examples may be provided by a server device. However, while illustrated and discussed as being provided by a server device, one of skill in the art in possession of the present disclosure will recognize that the functionality of the computing device 200 discussed below may be provided by other devices that are configured to operate similarly as the computing device 200 discussed below. In the illustrated embodiment, the computing device 200 includes a chassis 202 that houses the components of the computing device 200, only some of which are illustrated and described below.

For example, the chassis 202 may house a processing system (not illustrated, but which may include the processor 102 discussed above with reference to FIG. 1 such as a Central Processing Unit (CPU)) and a memory system (not illustrated, but which may include the memory 114 discussed above with reference to FIG. 1 such as Dynamic Random Access Memory (DRAM)) that is coupled to the processing system and that includes instructions that, when executed by the processing system, cause the processing system to provide an operating system engine 204 that is configured to perform the functionality of the operating system engines and/or computing devices discussed below. In the illustrated embodiment, the memory system may also include instructions that, when executed by the processing system, cause the processing system to provide a software RAID engine 204a that is configured to perform the functionality of the software RAID engines, software RAID subsystems, and/or computing devices discussed below.

To provide a specific example, the software RAID engine 204a may include a software RAID driver that is configured to utilize hardware resources in the computing device (e.g., the CPU, memory, etc.) in order to create and manage RAID infrastructure and/or perform any of a variety of RAID operations known in the art without the need for dedicated/specialized RAID hardware (e.g., a dedicated hardware RAID controller). However, while the software RAID engine 204a is illustrated and described as being included in the operating system and/or otherwise being provided by the operating system engine 204 (i.e., being provided by the processing/memory system combination that also provides the operating system engine 204), one of skill in the art in possession of the present disclosure will appreciate how the software RAID engine 204a may be provided separately from the operating system/operating system engine 204 while remaining within the scope of the present disclosure as well.

In an embodiment, the software RAID driver in the software RAID engine 204a will include a Small Computer

6

System Interface (SCSI)-based driver, and one of skill in the art in possession of the present disclosure will appreciate how such a SCSI-compliant driver may be configured to be utilized with any PCIe devices/PCIe controller devices (e.g., Advanced Host Controller Interface (AHCI) controllers, SAS controllers, virtual PCIe controllers, NVMe controllers, etc.), thus allowing a computing device manufacturer to provide a single SCSI-based software RAID driver on a plurality of different computing device configurations of computing devices manufactured by the computing device manufacturer in order to enable the software RAID functionality described below in any of those computing devices using the PCIe devices/PCIe controller devices included in those computing devices. To provide a specific example, the software RAID driver in the software RAID engine 204a may be implemented via the Storport/miniport model used in the WINDOWS® operating system, although other software RAID driver configurations will fall within the scope of the present disclosure as well.

In the illustrated embodiment, the memory system may also include instructions that, when executed by the processing system, cause the processing system to provide an operating system 204b that is configured to use and control the hardware resources in the computing device 200, and/or perform any of the other functionality of the operating systems and/or computing devices discussed below. For example, the operating system 204b may be provided by a WINDOWS® operating system available from MICROSOFT® Corp. of Redmond, Washington, United States, although other operating systems are envisioned as falling within the scope of the present disclosure as well.

In the illustrated embodiment, the chassis 202 also houses one or more controller devices 206 that are coupled to the software RAID engine 204a (e.g., via a coupling between the controller device(s) 206 and the processing system that provides the software RAID engine 204a). As discussed in the specific examples provided below, the controller device(s) 206 may be provided by Peripheral Component Interconnect express (PCIe) controller devices that may be included in and/or coupled to PCIe devices that are not provided by storage devices. For example, the PCIe controller device(s) 206 may be physical or virtual PCIe controller devices and may be included in Host Bus Adapter (HBA) devices, Advanced Host Controller Interface (AHCI) devices, and/or other PCIe devices that would be apparent to one of skill in the art in possession of the present disclosure. However, while illustrated and described as being provided by PCIe controller device(s), one of skill in the art in possession of the present disclosure will appreciate how the controller device(s) 206 may be provided by other types of controller devices while remaining within the scope of the present disclosure as well.

In the illustrated embodiment, the chassis 202 also houses one or more controller devices 208a that are coupled to the software RAID engine 204a (e.g., via a coupling between the controller device(s) 208a and the processing system that provides the software RAID engine 204a). As illustrated and discussed in the specific examples provided below, the controller device(s) 208a may be provided by Peripheral Component Interconnect express (PCIe) storage controller devices that are coupled to one or more storage devices 208b that may be provided by any of Non-Volatile Memory express (NVMe) storage devices, Serial Attached Small Computer System Interface (SCSI) (SAS) storage device, Serial AT Attachment (SATA) storage devices, and/or other storage devices that would be apparent to one of skill in the art in possession of the present disclosure. For example, the

PCIe storage controller device(s) **208a** may be provided by physical or virtual PCIe storage controller devices and may include PCIe storage controller devices configured as NVMe storage controllers, SAS storage controllers, SATA storage controllers, and/or other PCIe storage controller devices that would be apparent to one of skill in the art in possession of the present disclosure. However, while illustrated and described as being provided by PCIe controller device(s), one of skill in the art in possession of the present disclosure will appreciate how the controller device(s) **208a** may be provided by other types of controller devices while remaining within the scope of the present disclosure as well.

In the illustrated embodiment, the chassis **202** also houses one or more controller devices **210a** that are coupled to the software RAID engine **204a** (e.g., via a coupling between the controller device(s) **208a** and the processing system that provides the software RAID engine **204a**). As illustrated and discussed in the specific examples provided below, the controller device(s) **210a** may be provided by Peripheral Component Interconnect express (PCIe) controller devices that are included in one or more storage devices **210** that may be provided by any of Non-Volatile Memory express (NVMe) storage devices, Serial Attached Small Computer System Interface (SCSI) (SAS) storage device, Serial AT Attachment (SATA) storage devices, and/or other storage devices that would be apparent to one of skill in the art in possession of the present disclosure. For example, the PCIe storage controller device(s) **210a** may be provided by physical or virtual PCIe storage controller devices and may include PCIe storage controller devices configured as NVMe storage controllers. However, while illustrated and described as being provided by PCIe controller device(s), one of skill in the art in possession of the present disclosure will appreciate how the controller device(s) **210a** may be provided by other types of controller devices while remaining within the scope of the present disclosure as well.

However, while three different embodiments of controller devices **206**, **208a**, and **210a** are illustrated and described as being included in the computing device **200** (e.g., stand-alone controller device(s), storage-device-connected controller device(s), and storage-device-integrated controller device(s)), one of skill in the art in possession of the present disclosure will appreciate how one or more of the controller devices **206**, **208a**, and/or **210a** may be omitted from the computing device **200** while remaining within the scope of the present disclosure as well. Furthermore, while not explicitly illustrated, one of skill in the art in possession of the present disclosure will appreciate how any of the controller devices **208a** and/or **210a** may be coupled to and/or may control multiple storage devices **208b** and/or **210**, respectively, while remaining within the scope of the present disclosure as well.

Further still, one of skill in the art in possession of the present disclosure will appreciate how storage devices may be coupled to the software RAID engine **204a** via multiple controller devices (e.g., when an NVMe storage device with an integrated controller device is connected via an external controller device to the software RAID engine **204a**). As such, while a specific computing device **200** has been illustrated and described, one of skill in the art in possession of the present disclosure will recognize that computing devices (or other devices operating according to the teachings of the present disclosure in a manner similar to that described below for the computing device **200**) may include a variety of components and/or component configurations for providing conventional computing device functionality, as well as the direct-attached storage device software RAID

functionality discussed below, while remaining within the scope of the present disclosure as well.

For example, one of skill in the art in possession of the present disclosure will appreciate how the present disclosure describes the direct-attached storage device software RAID system of the present disclosure implemented to operate with an operating system such as the WINDOWS® operating system described above. However, at least some of the inventors of the present disclosure have also developed a direct-attached storage device software RAID system that may be implemented with a hypervisor subsystem (e.g., the VMWARE®n ESXi hypervisor available from VMWARE® Inc. of Palo Alto, California, United States) in order to allow software RAIDs to be provided using any types of direct-attached storage devices and any types of CPUs in a computing device similarly as described below, and that direct-attached storage device software RAID system is described in U.S. patent application Ser. No. 18/227,914, filed Jul. 29, 2023, the disclosure of which is incorporated herein by reference in its entirety.

Referring now to FIG. 3, an embodiment of a method **300** for providing a software Redundant Array of Independent Disk (RAID) using direct-attached storage devices in a computing device is illustrated. As discussed below, the systems and methods of the present disclosure provide for the presentation by a software RAID subsystem to an operating system of a single, primary controller device as being connected to a logical storage device provided by a plurality of physical storage devices, with the software RAID subsystem receiving commands from the operating system that are directed to the primary controller device and that identify a logical storage subsystem that is included in the logical storage device, and transmitting those commands to a subset of the physical storage devices that provide that logical storage subsystem via respective controller device(s) that couple the software RAID subsystem to that subset of physical storage devices.

For example, the direct-attached storage device software RAID system of the present disclosure may include a chassis housing a software RAID subsystem coupled to physical storage devices, controller devices, and an operating system housed in the chassis. The software RAID subsystem uses the physical storage devices to provide a logical storage device to the operating system. The software RAID subsystem also presents a first controller device to the operating system as a primary controller device that is connected to the logical storage device. When the software RAID subsystem receives a command from the operating system directed to the primary controller device and identifying a logical storage subsystem in the logical storage device, it transmits the command to each of a subset of the physical storage devices that provide the logical storage subsystem in the logical storage device via a respective one of the controller devices that couples the software RAID subsystem to that physical storage device. As such, software RAIDs may be provided using any types of direct-attached storage devices and any types of CPUs in a computing device, as well as provide a variety of other benefits that would be apparent to one of skill in the art in possession of the present disclosure.

The method **300** begins at block **302** where a software RAID subsystem configures a logical storage device using physical storage devices. With reference to FIG. 4, in an embodiment of block **302**, the software RAID engine **204a** in the computing device **200** may perform logical storage device configuration operations **400** that may include discovering each of the controller devices **206**, **208a**, and **210a** and, in response, discovering each of the storage devices

208b and/or **210** coupled thereto, and then configuring a logical storage device using those storage devices **208b** and/or **210**. In a specific example, each of the storage device(s) **208a** may be coupled to a PCIe controller device that is provided by one of the controller device(s) **208a** discovered at block **302**, and each of the storage device(s) **210** may include a PCIe controller device that is provided by one of the controller device(s) **210a** discovered at block **302**, while PCIe controller device(s) that are provided by the controller device(s) **206** and that are not coupled to storage devices may be discovered at block **302** as well.

For example, in response to the powering on, reset, reboot, and/or other initialization of the computing device **200**, the computing device **200** may enter a pre-boot mode in which the software RAID engine **204a** performs any of a variety of software RAID operations and/or other techniques that would be apparent to one of skill in the art in possession of the present disclosure in order to configure a RAID using the storage devices **208a** and **210** that allows the software RAID engine **204a** to present those storage devices **208a** and **210** as one or more logical storage devices. As will be appreciated by one of skill in the art in possession of the present disclosure, the RAID may be configured using the storage devices **208a** and **210** with a variety of standard RAID levels such as a RAID0, RAID1, RAID2, RAID3, RAID4, RAID5, or RAID 6, as well as combinations of standard RAID levels (e.g., RAID 1+0, also known as RAID10), non-standard RAID levels, and/or any other RAID levels that would be apparent to one of skill in the art in possession of the present disclosure.

The method **300** then proceeds to block **304** where the software RAID subsystem allocates memory space for a direct-attached storage device software RAID system. In an embodiment, at block **304**, the software RAID engine **204a** in the computing device **200** may perform memory space allocation operations that may include allocating memory space for use by the direct-attached storage device software RAID system of the present disclosure. For example, the memory space allocation operations performed by the software RAID engine **204a** may include allocating a memory space or other portion of the memory system that is used to provide the operating system engine **204** (e.g., operating system kernel memory) for use by the software RAID engine **204a** to perform any of the direct-attached storage device software RAID operations or other functionality discussed below. As will be appreciated by one of skill in the art in possession of the present disclosure, the memory space or other portion of the memory system allocated for use by the direct-attached storage device software RAID system may be referred to a “non-cached/non-coherent memory” that may be dynamically allocated during initialization of the computing device **200** to the software RAID driver provided by the software RAID engine **204a** (e.g., unlike memory space allocated to stacks), and one of skill in the art in possession of the present disclosure will recognize how the non-cached/non-coherent memory may be utilized to perform any of the functionality described below.

In a specific example, the allocation of the non-cached/non-coherent memory to the software RAID engine **204a** may be performed during module initialization operations that are performed when the operating system **204b** provided by the operating system engine **204** is loaded and calls those module initialization operations, and the software RAID engine **204a** may allocate the non-cached/non-coherent memory based on a number of storage devices (e.g., a maximum number of storage devices) that the software RAID engine **204a** is configured to support. However, while

a specific example of the allocation of memory space for use by the direct-attached storage device software RAID system of the present disclosure has been described, one of skill in the art in possession of the present disclosure will appreciate how memory space may be allocated for use by the direct-attached storage device software RAID system using a variety of techniques that will fall within the scope of the present disclosure as well.

The method **300** then proceeds to block **306** where the software RAID subsystem and the operating system register a controller device supported by the software RAID subsystem. With reference to FIG. 5, in an embodiment of block **306**, the controller device registration operations may include the software RAID engine **204a** in the computing device **200** performing supported controller device identification operations **500** that may include identifying controller devices that are supported by the software RAID engine **204a** to the operating system **204b** in the computing device **200** (e.g., using miniport driver commands). For example, any of the controller devices **206**, **208a**, and/or **210a** discovered at block **302** as discussed above may be identified to the operating system **204b** as supported controller devices at block **306**. In a specific example, at block **306**, the software RAID engine **204a** may generate a Peripheral Component Interconnect (PCI) IDentification (PCIID) file that identifies the PCIe controller devices that are supported by the software RAID engine **204a**, and may load or otherwise transmit that PCIID file to the operating system **204b**. However, while a specific example of the identification of controller devices to the operating system **204b** that are supported by the software RAID engine **204a** has been described, one of skill in the art in possession of the present disclosure will appreciate how the software RAID engine **204a** may identify supported controller devices to the operating system **204b** using other techniques that will fall within the scope of the present disclosure as well.

With reference to FIG. 6A, in an embodiment of block **306**, the controller device registration operations may also include the operating system **204b** in the computing device **200** performing controller device attachment request operations **600** that may include the operating system **204b** generating and transmitting a controller device attachment request for one of the controller devices that was identified as being supported by the software RAID engine **204a**. For example, the operating system **204b** may generate and transmit a “Find Adapter” call that provides the controller device attachment request and that one of skill in the art in possession of the present disclosure will recognize provides an instruction to the software RAID engine to find and attach one of the controller devices that was identified as being supported (e.g., via a Storport layer). However, while a specific example of a controller device attachment request has been described, one of skill in the art in possession of the present disclosure will appreciate how the operating system **204b** may request the initialization and discovery of controller devices using other techniques that will fall within the scope of the present disclosure as well.

With reference to FIG. 6B, in an embodiment of block **306** and in response to receiving the controller device attachment request (e.g., the “FindAdapter” call), the controller device registration operations may also include the software RAID engine **204a** and the operating system **204b** in the computing device **200** performing controller device attachment operations **602** that may include the software RAID engine **204a** processing the controller device attachment request, attaching a corresponding controller device, and confirming the completion of the controller device

11

attachment request that was received from the operating system **204b**. For example, in response to the successful attachment of a connected controller device and its corresponding storage device(s), the software RAID engine **204a** may confirm the attachment request to the operating system **204b** (e.g., via a “Find Adapter call success” communication transmitted to the operating system **204b**). While not illustrated or discussed in detail, one of skill in the art in possession of the present disclosure will appreciate how the software RAID engine **204a** may identify to the operating system **204b** any failures of the attachment of any connected controller devices and its corresponding storage device(s) as well. In response to receiving the confirmation of the controller device attachment request (e.g., the “Find Adapter call success” communication), the controller device attachment operations **602** may include the operating system **204b** generating and transmitting an initialization and discovery request (e.g., a “HwInitialize” call) to the software RAID engine **204a**.

With reference to FIG. 6C, in an embodiment of block **306** and in response to receiving the initialization and discovery request from the operating system **204b**, the controller device registration operations may also include the software RAID engine **204a** in the computing device **200** performing initialization and discovery operations **604** that may include, in the specific example provided in FIG. 6C, the software RAID engine **204a** initializing and discovering the one of the controller device(s) **206** identified by the operating system **204b** in the controller device attachment request. As will be appreciated by one of skill in the art in possession of the present disclosure, the initialization and discovery operations **604** performed by the software RAID engine **204a** may include any of a variety of initialization operations and discovery operations that would be apparent to one of skill in the art in possession of the present disclosure, that may depend on the protocol supported by that controller device (e.g., a SAS protocol, a SATA protocol, an NVMe protocol, etc.), and that result in the registration of that controller device **206** with the operating system **204b**.

In some embodiments, following the initialization and discovery of the first controller device during the first iteration of block **306**, the software RAID engine **204a** in the computing device **200** may perform timer registration operations that may include registering a timer with the operating system **204b**. As discussed below, some embodiments of the present disclosure may include the utilization of a timer to determine when the last controller device has been registered as part of the controller device registration operations performed across one or more iterations of block **306** of the method **300**, and thus that timer may be registered by the software RAID engine **204a** with the operating system **204b** at block **306** using any of a variety of timer registration techniques that would be apparent to one of skill in the art in possession of the present disclosure. However, the registration of the timer is described as occurring at a particular point in method **300** (e.g., following initialization and discovery of the first controller device during the first iteration of block **306**), one of skill in the art in possession of the present disclosure will appreciate how the timer may be registered at other points in the method **300** while remaining within the scope of the present disclosure as well. Furthermore, and as discussed in further details below, other embodiments of the method **300** may utilize other techniques to determine when the last controller device has been registered as part of the controller device registration operations across one or more iterations of block **306** of the

12

method **300**, and thus the timer registration operations may be skipped in those embodiments.

The method **300** then proceeds to decision block **308** where it is determined whether there is an additional controller device supported by the software RAID subsystem to register. In some embodiments of decision block **308**, following the registration of the one of the controller device(s) **206** (in the specific example provided in FIG. 6B) in the operating system **204b** at block **306**, the software RAID engine **204a** in the computing device **200** may perform timer activation operations in order to activate the timer discussed above that may have been registered with the operating system **204b** in some embodiments of the present disclosure. The software RAID engine **204a** may then monitor that timer at decision block **308** to determine whether another controller device attachment request (e.g., the “FindAdapter” call in the specific example provided above) for another of the controller devices **206**, **208b**, and/or **210a** is received from the operating system **204b** within a threshold time period. As will be appreciated by one of skill in the art in possession of the present disclosure, the software RAID engine **204a** may utilize any of a variety of threshold time periods that, after which, the software RAID engine **204a** may assume that no further controller device attachment requests will be received (i.e., that there are no further controller devices to be registered with the operating system **204b**).

However, while the use of a timer and threshold time period following registration of any particular controller device in order to determine whether there are any other controller devices to register at decision block **308** has been described, as discussed above other techniques for determining whether there are any other controller devices to register at decision block **308** will fall within the scope of the present disclosure as well. For example, other embodiments of the present disclosure may provide a software driver in the computing device **200** that is configured to identify a number of available controller devices in the computing device **200**, and then transmit that number of available controller devices to the software RAID engine **204a** for use in determining at decision block **308** whether the number of controller devices registered with the operating system **204a** has reached that number of available controller devices. As such, the determination of whether there are any other controller devices to register in the operating system **204b** during decision block **308** of the method **300** may be performed in a variety of manners while remaining within the scope of the present disclosure.

If, at decision block **308**, it is determined that there is an additional controller device supported by the software RAID subsystem to register, the method **300** returns to block **306**. As such, the method **300** may loop such that the software RAID engine **204a** and the operating system **204b** in the computing device **200** operate to register any additional controller device with the operating system **204b**. Thus, different iterations of block **306** and decision block **308** may cause the software RAID engine **204a** and the operating system **204b** to register each of the controller devices in the computing device **200** in the operating system **204b**. For example, for any remaining controller devices **206**, **208a**, and **210a**, the controller device attachment request operations **600**, controller device attachment operations **602**, and initialization and discovery operations **604** described above for block **306** may be performed for that controller device, and block **306** may repeat until it is determined that there are no additional controller devices to register with the operating system **204b**.

13

If, at decision block **308**, it is determined that there are no additional controller devices supported by the software RAID subsystem to register, the method **300** proceeds to block **310** where the software RAID subsystem creates one or more logical storage subsystems in the logical storage device. In an embodiment, at block **310** and in response to determining that there are no additional controller devices to register with the operating system **204b**, the software RAID engine **204a** in the computing device **200** may perform logical storage subsystem creation operations that may include creating one or more logical storage subsystems in the logical storage device provided by the storage devices **208b** and **210**. For example, with reference to FIG. 7, in some embodiments the logical storage subsystem creation operations may include the software RAID engine **204a** performing metadata retrieval operations **700** that may include retrieving metadata from the controller devices **208a** and **210a** coupled to each of the storage devices **208b** and **210** that are being used to provide the logical storage device.

As will be appreciated by one of skill in the art in possession of the present disclosure, the metadata retrieved from the controller devices **208a** and **210a** coupled to each of the storage devices **208b** and **210** may identify one or more logical storage subsystems (e.g., virtual disks, RAID volumes, RAID Logical Unit Numbers (LUNs), and/or logical storage subsystems known in the art) that will be provided by each of the storage devices **208b** and **210**, as well as any other information that one of skill in the art in possession of the present disclosure would recognize as providing for the creation of the logical storage subsystems in the logical storage device. As such, at block **310**, the logical storage subsystem creation operations performed by the software RAID engine **204a** may include using the metadata retrieved from the controller devices **208a** and **210a** coupled to each of the storage devices **208b** and **210** to “build”, generate, and/or otherwise create one or more logical storage subsystems (e.g., virtual disks, RAID volumes, RAID LUNs, and/or logical storage subsystems known in the art) in the logical storage device that is provided by the storage devices **208b** and **210**. However, while a specific example of the creation of logical storage subsystem(s) in a logical storage device has been described, one of skill in the art in possession of the present disclosure will appreciate how the logical storage subsystems may be created in the logical storage device using a variety of techniques that will fall within the scope of the present disclosure as well.

The method **300** then proceeds to block **312** where the software RAID subsystem configures command communications with the operating system. With reference to FIG. 8, in an embodiment of block **312**, the software RAID engine **204a** in the computing device **200** may perform command and communication configuration operations **800** that may include identifying a primary controller device from the plurality of controller devices **206**, **208a**, and **210a**. For example, the controller device **206**, **208a**, or **210a** identified as the primary controller device at block **312** may be a PCIe controller device that may have been designated by a computing device manufacturer to operate as the primary controller device based on that PCIe controller device being common to a variety of different configurations of computing devices provided by the computing device manufacturer (and thus being present in each of those computing devices/ computing device configurations if needed to implement the direct-attached storage device software RAID system of the present disclosure). In another example, the controller device **206**, **208a**, or **210a** identified as the primary control-

14

ler device at block **312** may be a PCIe controller device that is not configured to be “hot removed” from the computing device **200**. However, while a few specific examples of controller devices that may be identified as primary controller devices have been described, one of skill in the art in possession of the present disclosure will appreciate how any of a variety of criteria may be used to designate the controller device that may be identified as the primary controller device at block **312** while remaining within the scope of the present disclosure as well.

Following the command configuration operations at block **312**, the direct-attached storage device software RAID system completes initialization such that the RAID volume provided by the logical storage device is configured for use, and one of skill in the art in possession of the present disclosure will appreciate how a full RAID volume or partial RAID volume may be configured via the method **300** as described above, and may be used for runtime data, during boot/initialization, and/or for other RAID volume uses that would be apparent to one of skill in the art in possession of the present disclosure.

The method **300** then proceeds to block **314** where the software RAID subsystem presents one of the controller devices to the operating system as a primary controller device that is connected to the logical storage device. With reference to FIG. 9, in an embodiment of block **314** and following the completion of the initialization of the direct-attached storage device software RAID system, the software RAID engine **204a** may present, to the operating system **204b** in the computing device **200**, a primary controller device **900** (which as discussed above is provided by one of the controller devices **206**, **208a**, or **210a**) as being connected to a RAID logical storage device **902** that is provided using a plurality of physical storage devices **904a**, **904b**, and up to **904c** (which may be any of the storage devices **208a** and **210** registered with the operating system **204b** as discussed above).

To provide a specific example, one of the controller device(s) **206** that is not connected to a storage device may be presented to the operating system **204b** as being connected to the RAID logical storage device **902**, although one of skill in the art in possession of the present disclosure will appreciate how any one of the controller devices **208a** or **210a** may be presented to the operating system **204b** as being connected to the RAID logical storage device **902** while remaining within the scope of the present disclosure as well. As will be appreciated by one of skill in the art in possession of the present disclosure, any of the controller devices **206**, **208a**, and **210a** that are not presented to the operating system **204b** as being connected to the RAID logical storage device **902** may instead be presented to the operating system **204b** as secondary controller devices that are not connected to the RAID logical storage device **902** or any of the storage devices **904a-904c** (i.e., despite at least some of those controller devices being physically connected to those storage devices **904a-904c**).

As such, each of the logical storage subsystems (e.g., virtual disks, RAID volumes, RAID LUNs, and/or logical storage subsystems known in the art) provided by the RAID logical storage device **902** are exposed to the operating system **204b** via the primary controller device **900**. Thus, using some of the specific examples discussed above, each of the logical storage subsystems (e.g., virtual disks, RAID volumes, RAID LUNs, and/or logical storage subsystems known in the art) provided by storage devices **904a-904b** may be presented to a user of the operating system provided

15

by the operating system engine **204** as SCSI storage devices or otherwise being provided by SCSI storage devices.

The method **300** then proceeds to block **316** where the software RAID subsystem receives a command from the operating system that is directed to a primary controller device and that identifies a logical storage subsystem. With reference to FIG. 10, in an embodiment of block **316**, the operating system **204b** in the computing device **200** may perform command provisioning operations **1000** that include generating a command that identifies a logical storage subsystem provided by the RAID logical storage device **902**, and transmitting that command to the primary controller device **900** that is presented as being connected to the RAID logical storage device **902**, which one of skill in the art in possession of the present disclosure will appreciate will result that in that command being received by the software RAID engine **204a** in the computing device **200**. In an embodiment, the command transmitted by the operating system **204b** to the software RAID engine **204a** may include I/() commands such as a read command, a write command, and/or any other RAID commands that would be apparent to one of skill in the art in possession of the present disclosure.

Using the specific example provided above in which the software RAID engine **204a** is provided by a SCSI-based driver, any commands received at block **316** by the software RAID engine **204a** from the operating system **204b** may be SCSI commands. Furthermore, as discussed in some of the specific examples above, the commands received at block **316** by the software RAID engine **204a** from the operating system **204b** may be mapped to protocol queues in the operating system **204b** that are used to communicate with the software RAID engine **204a** provided by the SCSI-based driver, and thus the software RAID engine **206a** may receive those commands via the accessing of those protocols queues.

The method **300** then proceeds to block **318** where the software RAID subsystem transmits the command to a subset of physical storage devices that provide the logical storage subsystem via respective controller device(s) that couple the software RAID subsystem to that subset of physical storage devices. With continued reference to FIG. 10, in an embodiment of block **318**, the software RAID engine **204a** may perform command transmission operations **1000a** and **1000b** that, in the specific example illustrated in FIG. 10, includes transmitting the command received at block **316** to the storage device **904a** and the storage device **904c** via their respective controller device(s) (e.g., any of the controller device(s) **208a** and **210a**) based on those storage devices **904a** and **904c** providing the logical storage subsystem identified in the command received by the software RAID engine **204a** from the operating system **204b**. As will be appreciated by one of skill in the art in possession of the present disclosure, the commands **1000a** and **1000b** transmitted to the storage devices **904a** and **904c** may be provided by modified versions of the command **1000** received from the operating system **204b** (e.g., SCSI commands received from the operating system **204b** may be translated to a protocol supported by the controller device/storage device to which it is transmitted).

For example, in response to receiving the command from the operating system **204b** in the computing device **200**, the software RAID engine **204a** in the computing device **200** may determine which of the storage devices **904a-904c** provide the logical storage subsystem identified in that command (e.g., the storage devices **904a** and **904c** in this specific example). As discussed above, in some specific examples, protocol queues in the operating system **204b**

16

may be mapped to SAS, SATA, and/or NVMe protocol queues in the software RAID engine **204a**, and thus the software RAID engine **204a** may provide any commands received from the operating system **204b** in the protocol queues associated with the protocol utilized by the storage devices that provide the logical storage subsystem to which those commands are directed. As such, the command received by the software RAID engine **204a** from the operating system **204b** may be placed in a protocol queue associated with the protocol used by the storage device **904a**, as well as in a protocol queue associated with the protocol used by the storage device **904c**.

In embodiments in which the software RAID engine **204a** is provided by a SCSI-based driver and the commands received from the operating system **204b** are SCSI commands, in the event the storage devices **904a** and/or **904c** are SAS storage devices or otherwise use the SAS protocol, SCSI commands may be placed in a SAS protocol queue in the software RAID engine **204a** and then transmitted to the storage devices **904a** and/or **904c** via their respective controller device(s) (e.g., any of the controller device(s) **208a** and **210a**). However, in the event the storage devices **904a** and/or **904c** are SATA storage devices or otherwise use the SATA protocol, SCSI commands may be placed in a SATA protocol queue in the software RAID engine **204a**, translated using a SCSI translation layer in the software RAID engine **204a** (e.g., a SCSI-to-Frame Information Structure (FIS) translation), and then transmitted to the storage devices **904a** and/or **904c** via their respective controller device(s) (e.g., any of the controller device(s) **208a** and **210a**). Similarly, in the event the storage devices **904a** and/or **904c** are NVMe storage devices or otherwise use the NVMe protocol, SCSI commands may be placed in an NVMe protocol queue in the software RAID engine **204a**, translated using a SCSI translation layer in the software RAID engine **204a** (e.g., a SCSI-to-NVMe translation), and then transmitted to the storage devices **904a** and/or **904c** via their respective controller device(s) (e.g., any of the controller device(s) **208a** and **210a**).

However, while a few specific examples of the translation of commands received from the operating system **204b** before providing them to the storage devices **904a-904c** via their respective controller devices have been described, one of skill in the art in possession of the present disclosure will appreciate that other command translations may be performed while remaining within the scope of the present disclosure as well. For example, one of skill in the art in possession of the present disclosure will appreciate how a PCIe controller device connected to a SAS controller device will operate as a SAS controller that utilizes the SAS protocol, and thus no command translations may be needed in such situations. Similarly, a PCIe controller device connected to a SATA controller device will operate as a SATA controller that utilizes the SATA protocol, and thus SCSI-to-FIS translations may be needed in such situations.

The method **300** then returns to block **316**. As such, the method **300** may loop such that the software RAID engine **204a** in the computing device **200** receives commands from the operating system **204b** in the computing device **200** and transmits those commands to the storage devices **904a-904c** via their respective controllers as long as the storage devices **904a-904c** are being used to provide the RAID logical storage device **902** to the operating system **204b**. While not described in detail herein, in the event the storage devices **904a-904c** will no longer be used to provide the RAID logical storage device **902** to the operating system **204b**, the software RAID engine **204a** may disconnect the controller

device(s) **206**, **208a**, and **210a** from the operating system **204b**, as well as perform any other operations that would be apparent to one of skill in the art in possession of the present disclosure.

Thus, systems and methods have been described that provide for the presentation by a SCSI-based software RAID driver to an operating system of a single, primary PCIe controller device as being connected to a RAID logical storage device provided by a plurality of SAS, SATA, and/or NVMe physical storage devices, with the SCSI-based software RAID driver receiving I/O commands from the operating system that are directed to the primary PCIe controller device and that identify a RAID logical storage subsystem that is included in the RAID logical storage device, and transmitting those commands to a subset of the SAS, SATA, and/or NVMe physical storage devices that provide that RAID logical storage subsystem via respective PCIe controller device(s) that couple the SCSI-based software RAID driver to that subset of SAS, SATA, and/or NVMe physical storage devices. As such, software RAID support in operating systems like the WINDOWS® operating system discussed above is enabled in direct-attached storage device configurations for any types of storage devices and any types of CPUs, thus allowing the creation of a RAID using any available storage devices in a variety of computing devices. As such, RAID systems may be configured from a larger variety of storage devices and using spanned PCIe controllers, addressing several issues with the conventional RAID systems discussed above.

However, the inventors of the present disclosure have recognized that the presentation to the WINDOWS® operating system of the single, primary controller device raises some issues with the operation of the direct-attached storage device software RAID system. For example, the WINDOWS® operating systems in server devices include “crash dump” functionality that, in response a “crash”, fault, unavailability, or other system issue in the server device, perform crash dump operations that include automatically generating crash dump information related to that system issue (e.g., the processes, drivers, programs, applications, and/or other components running or otherwise operating at the time of the system issue, the Kernel-mode stack that “stopped” or otherwise became unavailable, etc.), storing that crash dump information in a crash dump storage location, and subsequently providing access to that crash dump information in the crash dump storage location to help troubleshoot the cause of the system issue.

However, in server devices providing the direct-attached storage device software RAID system discussed above, the WINDOWS® operating system will be installed on a boot logical storage subsystem (e.g., a boot Logical Unit Number (LUN) in the logical storage device that will be exposed via a “boot controller device” provided by the primary controller device discussed above. When entering a crash dump mode following a crash, the WINDOWS® operating system will load a “Storport” driver to communicate with the boot controller device/primary controller device via a boot controller miniport driver that was also loaded by the WINDOWS® operating system. However, the operating system will not load miniport drivers for any of the secondary controller devices. As such, when the boot logical storage subsystem is provided by any storage devices controlled by the secondary controller devices (e.g., when the boot LUN is a virtual drive provided by a Non-Volatile Memory express (NVMe) storage device controlled by a secondary controller device, by a spanned virtual drive provided by multiple storage devices connected to different secondary

controller devices, etc.), those storage devices will not be initialized (i.e., due to the lack of loading of miniport drivers for their secondary controller devices), and the crash dump information will not be generated.

Referring now to FIG. 11, a method **1100** for providing crash dumps in a software Redundant Array of Independent Disk (RAID) using direct-attached storage devices in a computing device is illustrated. As discussed below, the systems and methods of the present disclosure provide for the initialization of secondary controller device(s) that control storage device(s) that provide a boot logical storage subsystem while in a crash dump mode. For example, the direct-attached storage device software RAID crash dump system of the present disclosure may include a chassis housing a software RAID subsystem coupled to each of a plurality of physical storage devices, each of a plurality of controller devices, and an operating system. While in a runtime mode, the software RAID subsystem presents the operating system with a primary controller device from the plurality of controller devices as being connected to a logical storage device provided by the plurality of physical storage devices. In response to entering a crash dump mode from the runtime mode, the software RAID subsystem provides a storage driver subsystem for the primary controller device and uses it to initialize at least one secondary controller device in the plurality of controller devices that controls at least one of the plurality of physical storage devices that provides a boot logical storage subsystem in the logical storage device. As such, crash dumps for operating systems like the WINDOWS® operating system discussed above are enabled in the direct-attached storage device software RAID system of the present disclosure.

The method **1100** begins at block **1102** where a software RAID subsystem provides storage driver subsystems for a primary controller device and secondary controller devices. With reference to FIG. 12, an embodiment of a computing device **1200** that may be provided by the computing device **200** of FIG. 2 is illustrated for use in the discussions below. As can be seen, computing device **1200** includes the software RAID engine **204a** and the operating system **204b** discussed above with reference to FIG. 2. In addition, the computing device **1200** may include a storage device (not illustrated, but which may be provided by a storage device similar to the storage device **108** discussed above with reference to FIG. 1) that is accessible to the software RAID engine **204a** and the operating system **204b** (e.g., via a coupling between the storage device and the processing system(s) that provides the software RAID engine **204a** and the operating system engine **204b**) and that includes a controller initialization database **1201**.

Furthermore, the computing device **1200** also includes a plurality of storage devices **1202a**, **1202b**, and up to **1202c**, each of which may be provided by the storage devices **208b** and/or **210** discussed above, and one of skill in the art in possession of the present disclosure will appreciate how those storage devices **1202a-1202c** may be configured to provide a logical storage device similarly as described above. As illustrated, a primary controller device **1204a** is connected to the storage device(s) **1202a** and may be provided by any of the controller devices **206**, **208a**, and **210a**; a secondary controller device **1204b** is connected to the storage device(s) **1202b** and may be provided by any of the controller devices **206**, **208a**, and **210a**; and up to a secondary controller device **1204c** is connected to the storage device(s) **1202c** and may be provided by any of the controller devices **206**, **208a**, and **210a**, and one of skill in the art in possession of the present disclosure will recognize

that the primary controller device **1204a** and the secondary controller devices **1204b-1204c** may be designated as “primary” and “secondary” according to the teachings of the present disclosure described above. Furthermore, as discussed above, the primary controller device **1204a** need not be connected to storage devices while remaining within the scope of the present disclosure as well.

Finally, in the examples below, at least one of the storage devices **1202b** and at least one of the storage devices **1202c** is configured to provide a boot logical storage subsystem (e.g., a boot LUN) on the logical storage device provided by the storage devices **1202a-1202c**. However, while a specific example is provided, one of skill in the art in possession of the present disclosure will appreciate how only a single secondary controller device may be connected to storage device(s) that provide the boot logical storage subsystem on the logical storage device while remaining within the scope of the present disclosure as well.

In an embodiment, at block **1102** and during a runtime mode for the computing device **1200** (i.e., when the operating system **204b** controls the computing device **1200**), the software RAID engine **204a** may provide a respective software driver engine for each of the storage controller devices, with the illustrated embodiment including the software RAID engine **204a** providing a software driver engine **1206a** for the primary storage controller device **1204a**, a software driver engine **1206b** for the secondary storage controller device **1204b**, and up to a software driver engine **1206c** for the secondary storage controller device **1204c**. For example, the software RAID engine **204a** may provide instructions on a memory system in each of the primary controller device **1204a** and the secondary controller devices **1204b-1204c** that, when executed by a processing system on each of the primary controller device **1204a** and the secondary controller devices **1204b-1204c**, cause those processing systems to provide the storage driver engines **1206a** and **1206b-1206c**, respectively, that are configured to perform the functionality of the storage driver engines, storage driver subsystems, and/or storage drivers discussed below.

As such, while the software driver engines **1206a** and **1206b-1206c** are illustrated in FIG. 12 as being “included in” the software RAID engine **204a**, one of skill in the art in possession of the present disclosure will appreciate how the storage driver engines **1206a** and **1206b-1206c** may be provided by processing system/memory system combinations in the primary controller device **1204a** and the secondary controller devices **1204b-1204c**, respectively. In a specific example, the storage driver engines **1206a** and **1206b-1206c** may be configured to provide respective mini-port drivers for each of the primary controller device **1204a** and the secondary controller devices **1204b-1204c**, respectively, although one of skill in the art in possession of the present disclosure will recognize how other storage driver engines will fall within the scope of the present disclosure as well.

The method **1100** then proceeds to block **1104** where the software RAID subsystem uses the storage device subsystems provided for the secondary controller devices to provide secondary controller initialization information in a controller initialization database. With reference to FIG. 13, in an embodiment of block **1104** and during the runtime mode for the computing device **1200** (i.e., when the operating system **204b** controls the computing device **1200**), the storage driver engines **1206b-1206c** may perform secondary controller initialization information provisioning operations **1300** that include generating secondary controller initialization information for their respective secondary controller

devices **1204b-1204c**, and transmitting that secondary controller initialization information for storage in the controller initialization database **1201**. As will be appreciated by one of skill in the art in possession of the present disclosure, at block **1104** the software RAID engine **204a** may identify that the secondary controller devices **1204b-1204c** control storage devices **1202b-1202c** that store the boot logical storage subsystem (e.g., the boot LUN), and may cause the storage driver engines **1206b-1206c** provided for those secondary controller devices **1204b-1204c** to provide corresponding secondary controller initialization information in the controller initialization database **1201**.

For example, and as discussed below, the direct-attached storage device software RAID crash dump system of the present disclosure may define a “CONFIG VALUES” structure that is configured to hold any secondary controller initialization information that is necessary to initialize a secondary controller device, a mapped memory address for initializing a secondary controller device, and/or any other secondary controller initialization information that one of skill in the art in possession of the present disclosure would appreciate as allowing for the secondary controller initialization functionality described below. In a specific example, the secondary controller initialization information provisioning operations **1300** at block **1104** of the method **1100** may include each of the storage driver engines **1206b-1206c** generating secondary controller initialization information in the respective “CONFIG VALUES” structure for their respective secondary controller devices **1204b-1204c** that may include Base Address Register (BAR) information for that secondary controller device, Bus: Device: Function (BDF) information for that secondary controller device, and/or any of a variety of other information (e.g., Peripheral Component Interconnect express (PCIe) information) that would be apparent to one of skill in the art in possession of the present disclosure.

Furthermore, the secondary controller initialization information provisioning operations **1300** at block **1104** of the method **1100** may include each of the storage driver engines **1206b-1206c** updating the “CONFIG VALUES” structure for their respective secondary controller devices **1204b-1204c** in a global “SWRAID_DUMP_CONTEXT” structure that is stored in the controller initialization database **1201** and that may be configured to store an array of the “CONFIG VALUES” structures, a variable to track a count of the valid entries in the array, and/or any other information that one of skill in the art in possession of the present disclosure would recognize as providing the functionality described below.

In addition, the secondary controller initialization information provisioning operations **1300** at block **1104** of the method **1100** may include each of the storage driver engines **1206b-1206c** reserving a memory space in a memory system of the computing device **1200** for use in initializing its respective secondary controller device **1204b-1204c** during a crash dump mode for the computing device **1200**, and transmitting a memory space identifier (e.g., a pointer) for that memory space for storage in the global “SWRAID_DUMP_CONTEXT” structure in the controller initialization database **1201**. As such, following block **1104**, the global “SWRAID_DUMP_CONTEXT” structure in the controller initialization database **1201** may store the “CONFIG VALUES” structure for each of the secondary controller devices **1204b-1204c** in association with a memory space identifier for a memory space that will be used to initialize that secondary controller device during a crash dump mode for the computing device **1200**. However, while a specific

21

example of the provisioning of secondary controller initialization information has been described, one of skill in the art in possession of the present disclosure will appreciate how a variety of secondary controller initialization information may be provided in a variety of manners while remaining within the scope of the present disclosure as well.

The method 1100 then proceeds to block 1106 where the software RAID subsystem uses the storage driver subsystem provided for the primary controller device to provide a controller initialization database identifier to an operating system. With reference to FIG. 14, in an embodiment of block 1106 and during the runtime mode for the computing device 1200 (i.e., when the operating system 204b controls the computing device 1200), the storage driver engine 1206a provided for the primary controller device 1204a may perform controller initialization database identifier provisioning operations 1400 with the operating system 204b in order to provide a controller initialization database identifier for the controller initialization database 1201 to the operating system 204b.

For example, the controller initialization database identifier provisioning operations 1400 performed by the operating system 204b at block 1106 may include the operating system 204b transmitting a Storage Request Block (SRB) communication to the storage driver engine 1206a provided for the primary controller device 1204a. In a specific example, the SRB communication transmitted by the operating system 204b to the storage driver engine 1206a at block 1106 may include an “SRB_FUNCTION_DUMP_POINTERS” function code that one of skill in the art in possession of the present disclosure will appreciate begins the configuration of the storage driver engine 1206a to access the controller initialization database 1201 (e.g., the global “SWRAID_DUMP_CONTEXT” structure discussed above) when the computing device 1200 enters the crash dump mode as described below. In response to receiving the SRB communication from the operating system 204b at block 1106, the storage driver engine 1206a may transmit a controller initialization database identifier (e.g., a pointer) for the controller initialization database 1201 to the operating system 204b. To provide a specific example, the controller initialization database identifier may include an address of the global “SWRAID_DUMP_CONTEXT” structure discussed above. However, while a specific example of providing a controller initialization database identifier to an operating system has been described, one of skill in the art in possession of the present disclosure will appreciate how controller initialization information and/or controller initialization databases may be identified to operating systems in a variety of manners that will fall within the scope of the present disclosure as well.

The method 1100 then proceeds to decision block 1108 where the method 1100 proceeds depending on whether the software RAID subsystem has entered a crash dump mode. As discussed above, the operating system 204b (e.g., a WINDOWS® operating system discussed above) may be configured with “crash dump” functionality that, in response to a “crash”, fault, unavailability, or other system issue in the computing device 1200, cause the computing device 1200 to enter a crash dump mode from the runtime mode discussed above, and then perform crash dump operations that include automatically generating crash dump information related to that system issue (e.g., the processes, drivers, programs, applications, and/or other components running or otherwise operating at the time of the system issue, the Kernel-mode stack that “stopped” or otherwise became unavailable, etc.), storing that crash dump information in a crash dump storage

22

location, and subsequently providing access to that crash dump information in the crash dump storage location to help troubleshoot the cause of the system issue. As such, at decision block 1108, the method 1100 will proceed depending on whether or not the computing device 1200 crashes and enters the crash dump mode. If, at decision block 1108, the computing device 1200 does not crash and thus does not enter the crash dump mode, the method 1100 returns to decision block 1108. Thus, the method 1100 may loop until the computing device 1200 crashes and enters the crash dump mode.

If, at decision block 1108, the computing device 1200 crashes and enters the crash dump mode, the method 1100 then proceeds to block 1110 where the software RAID subsystem provides a storage driver subsystem for the primary controller device. With reference to FIG. 15, in an embodiment of block 1110 and in response to entering the crash dump mode, the software RAID engine 204a may provide a storage driver engine 1500 for the primary controller device 1204a, and as illustrated in FIG. 15 and discussed above, the software RAID engine 204a may be configured to not provide storage driver engines for any of the secondary controller devices 1204b-1204c. For example, in response to entering the crash dump mode, the software RAID engine 204a may provide the software driver engine 1500 that is similar to the storage driver engine 1406a discussed above but that is configured to provide a specialized “boot controller” miniport driver for the primary controller device 1204a, which as discussed above is presented as being connected to the logical storage device provided by the storage devices 1202a-1202c and thus is presented as being connected to the boot logical storage subsystem (e.g., a boot LUN) in that logical storage device.

In addition, one of skill in the art in possession of the present disclosure will appreciate how the operating system 204b (e.g., a reduced functionality version of the operating system 204b relative to its functionality while in the runtime mode) may provide a “Storport” driver (e.g., a special copy of the “Storport” provide called a “storport dump” driver) for communicating with the “boot controller” miniport driver (e.g., a special copy of the “miniport” driver called a “miniport dump” driver) that is also provided by the operating system 204b while in crash dump mode. However, while a specific example of a storage driver subsystem has been described, one of skill in the art in possession of the present disclosure will appreciate how other storage driver engines may fall within the scope of the present disclosure as well.

The method 1100 then proceeds to block 1112 where the software RAID subsystem uses the storage driver subsystem provided for the primary controller device to initialize one or more secondary controller device(s) that control physical storage device(s) that provide a boot logical storage subsystem in a logical storage device. With reference to FIG. 16, in an embodiment of block 1112 and during the crash dump mode for the computing device 1200, the operating system 204b may perform controller initialization database identification operations 1600 that may include identifying the controller initialization database 1201 to the storage driver engine 1500 provided for the primary controller device 1204a. For example, while in the crash dump mode, the operating system 204b may call a “HwMiniportinitialize” function for the primary controller device 1204a that is presented to the operating system 204b as providing the boot logical storage subsystem (e.g., the boot LUN) in the logical storage device, with that “HwMiniportinitialize” function

23

operating to initialize the storage driver engine **1500** for the primary controller device **1204a**.

In a specific example, the controller initialization database identification operations **1600** at block **1112** may include the operating system **204b** providing the controller initialization database identifier (e.g., an address to the global “SWRAID_DUMP_CONTEXT” structure in the controller initialization database **1201**) that it received at block **1106** in the “HwMiniportinitialize” function by, for example, specifying that controller initialization database identifier in a “MiniportDumpData” member of a “PORT CONFIGURATION INFORMATION” structure in the “HwMiniportinitialize” function. However, while a specific example of the provision of a controller initialization database identifier to a storage driver engine while in a crash dump mode has been described, one of skill in the art in possession of the present disclosure will appreciate how the controller initialization database identifier may be provided to the storage driver engine while in the crash dump mode using other techniques that will fall within the scope of the present disclosure as well.

With reference to FIG. 17, in an embodiment of block **1112** and in response to receiving the controller initialization database identifier during the crash dump mode for the computing device **1200**, the storage driver engine **1500** may perform controller initialization information retrieval operations **1700** that may include retrieving the secondary controller initialization information that was provided in the controller initialization database **1201** at block **1104**. For example, at block **1112**, the storage driver engine **1500**/primary controller device **1204a** may identify the controller initialization database identifier (e.g., an address to the global “SWRAID_DUMP_CONTEXT” structure in the controller initialization database **1201**) in the “HwMiniportinitialize” function received from the operating system **204b**, and use that controller initialization database identifier to retrieve the secondary controller initialization information from the controller initialization database **1201** (e.g., the “CONFIG VALUES” structure for each of the secondary controller devices **1204b-1204c**, the associated memory space identifiers for the memory spaces that will be used to initialize those secondary controller devices **1204b-1204c**, and/or other secondary controller initialization information from the global “SWRAID_DUMP_CONTEXT” structure). However, while a specific example of the retrieval of secondary controller initialization information has been described, one of skill in the art in possession of the present disclosure will appreciate how a variety of secondary controller initialization information may be retrieved using a variety of techniques while remaining within the scope of the present disclosure as well.

With reference to FIG. 18, in an embodiment of block **1112** and during the crash dump mode for the computing device **1200**, the storage driver engine **1500** may perform secondary controller initialization operations **1800** that may include initializing each of the secondary controller devices **1204b-1204c**. For example, one of skill in the art in possession of the present disclosure will appreciate how the storage driver engine **1500** may use the BAR information, the BDF information, and/or other information (e.g., PCIe information) in the “CONFIG VALUES” structure for each of the secondary controller devices **1204b-1204c**, along with the respective memory spaces that were reserved for the initialization of those secondary controller devices **1204b-1204c**, in order to initialize each of those secondary controller devices **1204b-1204c** at block **1112** (e.g., via the programming of registers in those secondary controller

24

device that are identified by the BAR information, as well as any other initialization operations that would be apparent to one of skill in the art in possession of the present disclosure).

The method **1100** then proceeds to block **1114** where the software RAID subsystem performs crash dump operations to generate and store crash dump information associated with the boot logical storage subsystem. In an embodiment, at block **1114**, the operating system **204b** may perform any of a variety of crash dump operations that may include accessing the boot logical storage subsystem (e.g., the boot LUN) provided by any of the storage devices **1202b-1202c** controlled by the secondary controller devices **1204b-1204c** in order to retrieve crash dump information, which one of skill in the art in possession of the present disclosure will appreciate is possible while in the crash dump mode due to the initialization of those secondary controller devices **1204b-1204c** as described above.

Thus, systems and methods have been described that provide for the initialization of secondary controller device(s) that control storage device(s) that provide a boot logical storage subsystem while in a crash dump mode. For example, the direct-attached storage device software RAID crash dump system of the present disclosure may include a chassis housing a software RAID subsystem coupled to each of a plurality of physical storage devices, each of a plurality of controller devices, and an operating system. While in a runtime mode, the software RAID subsystem presents the operating system with a primary controller device from the plurality of controller devices as being connected to a logical storage device provided by the plurality of physical storage devices. In response to entering a crash dump mode from the runtime mode, the software RAID subsystem provides a storage driver subsystem for the primary controller device and uses it to initialize at least one secondary controller device in the plurality of controller devices that controls at least one of the plurality of physical storage devices that provides a boot logical storage subsystem in the logical storage device. As such, crash dumps for operating systems like the WINDOWS® operating system discussed above are enabled in the direct-attached storage device software RAID system of the present disclosure and with virtual drives provided by NVMe storage devices, spanned virtual drives provided by multiple storage devices, and/or in other situations that are not supported by conventional crash dump systems.

The crash dump data collected during the crash dumps discussed above will depend on the type of crash dump that the operating system is configured to perform (e.g., a “mini” crash dump, a “full” crash dump, a “kernel” crash dump, and/or other crash dump types that may be selected by a user). As will be appreciated by one of skill in the art in possession of the present disclosure, the size of full crash dumps is relatively large and can pose challenges with regard to the transfer of their corresponding crash dump data for analysis as well as the expertise needed to perform that analysis, while mini crash dumps may not include the crash dump data needed to perform debugging and/or other analysis operations. Furthermore, while the direct-attached storage device software RAID crash dump system described above provides for the storage of a variety of crash dump data generated and/or retrieved by the operating system in response to a crash, there is some RAID data that is not stored as part of conventional crash dumps.

For example, RAID driver telemetry data that may be generated by the software RAID engines (e.g., software RAID drivers) discussed above includes a collection of data that is useful in debugging issues with the software RAID

25

engine, and is not conventionally collected and stored by the operating system as part of crash dump operations (e.g., such RAID driver telemetry data is typically stored in RAID driver logs in the software RAID driver internal memory that is ignored by the operating system during the crash dump operations discussed above). As such, RAID driver telemetry data will be lost in the event of a crash such that it is not retrievable following recovery from the crash. Furthermore, the software RAID engine may also utilize a write-back cache subsystem (which may be provided by a memory system in the computing device) that may be used in writing data to the storage devices, and write data stored in such write-back cache subsystems that has not yet been “flushed” or otherwise written to the storage devices is not conventionally stored by the operating system as part of crash dump operations, and will be lost in the event of a crash. However, while two specific examples of RAID data that is not collected in conventional crash dump systems have been described, one of skill in the art in possession of the present disclosure will appreciate how other conventionally-uncollected RAID data may be stored as part of the crash-based RAID data storage functionality discussed below while remaining within the scope of the present disclosure as well.

With reference to FIG. 19, a method 1900 for storing RAID data in response to a crash of a computing device is illustrated. As discussed below, the systems and methods of the present disclosure reserve RAID driver telemetry portions of RAID logical storage subsystem in storage devices for use in storing RAID driver telemetry data in the event of a crash. For example, the crash-based RAID data storage system of the present disclosure may include a chassis housing storage subsystems coupled to an operating system and a software RAID subsystem. During a runtime mode of the operating system, the software RAID subsystem reserves a RAID driver telemetry storage portion of a RAID logical storage subsystem provided by at least one storage device in the storage subsystems. In response to the operating system entering a crash dump mode from the runtime mode, the software RAID subsystem initializes respective controller devices in each of the storage subsystems, and during the crash dump mode of the operating system the software RAID subsystem stores RAID driver telemetry data generated by the software RAID subsystem in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device. As such, RAID driver telemetry data that would otherwise be lost in response to a crash is saved and available for use in de-bugging and other analysis operations post-recovery from the crash.

As will be appreciated by one of skill in the art in possession of the present disclosure, the methods 1100 and 1900 may be performed together in order to enhance the crash dump functionality of the method 1100 via the crash-based RAID data storage functionality of the method 1900, and one of skill in the art in possession of the present disclosure will recognize how the methods 1100 and 1900 include common operations (e.g., determining whether a crash dump mode has been entered, initializing controllers, etc.) that may be performed once by the software RAID subsystem as part of both of the methods 1100 and 1900.

Referring now to FIG. 20, an embodiment of a computing device 2000 that may be provided by the computing device 200 discussed above with reference to FIG. 2 and/or the computing device 1200 discussed above with reference to FIG. 12 is illustrated. The computing device 2000 includes

26

a chassis 2002 that houses the components of the computing device 200, only some of which are illustrated and described below. As can be seen, the chassis 2002 may house the processing system and memory system that provide the software RAID engine 204a similarly as described above. Furthermore, in the embodiments illustrated and described below, the software RAID engine 204a also includes a write-back cache subsystem 2001 that one of skill in the art in possession of the present disclosure will appreciate may include a cache device (e.g., provided by part of the memory system that provides the software RAID engine 204a, a separate memory system, etc.) that is configured for use in optimizing data writes by temporarily storing data prior to “flushing” or otherwise writing that data in order to improve performance via the buffering data writes, the reduction of the number of data writes, and/or providing other performance improvements that would be apparent to one of skill in the art in possession of the present disclosure.

The chassis 2002 may also house a plurality of storage subsystems that each include a respective storage device 2002a, 2002b, and up to 2002c, any of which may be provided by any of the storage devices 208b, 210, 1202a, 1202b, and/or 1202c discussed above. Each of the storage devices 2002a-2002c are coupled to the software RAID engine 204a (e.g., via a coupling between that storage device and the processing system), and while not illustrated, one of skill in the art in possession of the present disclosure will appreciate how the storage subsystems that include the storage devices 2002a-2002c may include one or more controller devices that may be provided by any of the controller devices 208a, 210a, 1204a, 1204b, and/or 1204c discussed above that couple their storage device(s) to the software RAID engine 204a. However, while a specific computing device 2000 has been illustrated and described, one of skill in the art in possession of the present disclosure will appreciate how the computing device 2000 may include a variety of components and/or component configurations for providing conventional computing functionality, as well as the crash-based RAID data storage functionality described below, while remaining within the scope of the present disclosure as well.

With reference to FIG. 21, an embodiment of a storage device 2100 is illustrated that may provide any of the storage devices 2002a-2002c discussed above with reference to FIG. 20. As such, the storage device 2100 may be provided by any of the controller devices 208a, 210a, 1204a, 1204b, and/or 1204c discussed above as well. In the illustrated embodiment, the storage device 2100 includes a chassis 2102 that houses the components of the storage device 2100, only some of which are illustrated and described below. For example, the chassis 2002 may house a processing system (not illustrated, but which may include a processor provided in storage firmware) and a memory system (not illustrated, but which may include memory provided in storage firmware) that includes instructions that, when executed by the processing system, cause the processing system to provide a storage engine 2104 that is configured to perform the functionality of the storage engines and/or storage devices described below.

The chassis 2102 may also house a storage subsystem 2106 that is coupled to the storage engine 2104 (e.g., via a coupling between the storage subsystem 2106 and the processing system) and that may include any storage elements (e.g., NAND flash storage elements in a Non-Volatile Memory express (NVMe) storage device, etc.) that one of skill in the art in possession of the present disclosure will appreciate may be used by the storage device 2100 for the

mass storage of data. The chassis **2102** may also house a communication system **2108** that is coupled to the storage engine **2104** (e.g., via a coupling between the communication system **2108** and the processing system) and that may be provided by any of a variety of storage device communication components that would be apparent to one of skill in the art in possession of the present disclosure. However, while a specific storage device **2100** has been illustrated and described, one of skill in the art in possession of the present disclosure will appreciate how the storage device **2100** may include a variety of components and/or component configurations for providing conventional storage device functionality, as well as the crash-based RAID data storage functionality described below, while remaining within the scope of the present disclosure as well.

The method **1900** begins at block **1902** where a software RAID subsystem reserves a RAID driver telemetry portion of at least one RAID logical storage subsystem provided by at least one storage device in a plurality of storage subsystems during a runtime mode. In an embodiment, during or prior to the method **1900**, the computing device **2000** may be powered on, started up, reset, and/or otherwise initialized such that an initialization subsystem in the computing device **2000** (not illustrated, but which may be provided by a Basic Input/Output System (BIOS), Unified Extensible Firmware Interface (UEFI), etc.) initializes the computing device **2000** such that an operating system in the computing device **2000** (not illustrated, but which may be provided by the operating system **204b** described above) enters a runtime mode and takes control of the computing device **2000** (e.g., takes control of the hardware devices in the computing device **2000**).

With reference to FIG. **22A**, in an embodiment of block **1902** and during the runtime mode of the operating system/ computing device **2000**, the software RAID engine **204a** in the computing device **2000** may perform RAID driver telemetry portion reservation operations **2200** that may include reserving a RAID driver telemetry portion of the storage subsystem **2106** in each of the storage devices **2002a-2002c/2100**. For example, with reference to FIG. **22B**, the storage subsystem **2106** in the storage device **2100** is illustrated subsequent to the RAID driver telemetry portion reservation operations **2200** that have operating to configure a RAID logical storage subsystem **2202** provided by the storage subsystem **2202** with a RAID metadata section **2202a**, a user data section **2202b**, and a RAID driver telemetry data section **2202c** that was reserved in the RAID logical storage subsystem **2202** as part of the RAID driver telemetry portion reservation operations **2200**. However, while each of the storage devices **2002a-2002c/2100** are illustrated and described as having been configured with the RAID driver telemetry data section **2202c** in their RAID logical storage subsystem **2202** provided by their storage subsystem **2106**, one of skill in the art in possession of the present disclosure will appreciate how not all (and in some cases, only one) of the storage devices **2002a-2002c/2100** may be configured with the RAID logical driver telemetry data section **2202c** while remaining within the scope of the present disclosure as well.

The method **1900** then proceeds to decision block **1904** where it is determined whether a crash dump mode has been entered in response to a crash. As discussed above, the operating system **204b** (e.g., a WINDOWS® operating system discussed above) may be configured with “crash dump” functionality that, in response a “crash”, fault, unavailability, or other system issue in the computing device **2000**, cause the computing device **2000** to enter a crash

dump mode from the runtime mode discussed above, and then perform crash dump operations that include automatically generating crash dump information related to that system issue, storing that crash dump information in a crash dump storage location, and subsequently providing access to that crash dump information in the crash dump storage location to help troubleshoot the cause of the system issue. As such, at decision block **1904**, the method **1900** will proceed depending on whether the computing device **2000** crashes and enters the crash dump mode. If, at decision block **1904**, the computing device **2000** does not crash and thus does not enter the crash dump mode, the method **1900** returns to decision block **1904**. Thus, the method **1900** may loop until the computing device **2000** crashes and enters the crash dump mode.

If, at decision block **1904**, the computing device **2000** crashes and enters the crash dump mode, the method **1900** then proceeds to block **1906** where the software RAID subsystem initializes controller devices in the plurality of storage subsystems. As discussed above, the methods **1100** and **1900** may be performed together to provide the functionality of both, and thus blocks **1102**, **1104**, and **1106** of the method **1100** may have previously been performed in order to configure the initialization of the controller devices **1204a-1204b**, and both decision block **1108** of the method **1100** and decision block **1904** of the method **1900** may have been performed to determine that the computing device **1200/2000** has entered the crash dump mode.

As such, in an embodiment of block **1906**, blocks **1110** and **1112** of the method **1100** may be performed to provide the storage driver subsystem for the primary controller device, and use that storage driver subsystem to initialize secondary controller devices that control physical storage device(s) that provide a boot logical storage subsystem in the logical storage device, which one of skill in the art in possession of the present disclosure will appreciate operates to initialize the controller devices for the storage devices **1202a-1202c**. While not described in detail below, block **1114** of the method **1100** may be performed anytime during or after the remainder of the method **1900** to perform the crash dump operations described above that generate and store crash dump information associated with the boot logical storage subsystem.

The method **1900** then proceeds to block **1908** where the software RAID subsystem stores RAID driver telemetry data generated by the software RAID subsystem in the RAID driver telemetry storage portion of the at least one RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems. With reference to FIGS. **23A** and **23B**, in an embodiment of block **1908** and in response to entering the crash dump mode, the software RAID engine **204a** may perform RAID driver telemetry data storage operations **2300** that may include storing RAID driver telemetry data that was generated by the software RAID engine **204a** (e.g., and stored a memory system that provides the software RAID engine **204a**) in the RAID driver telemetry data section **2202c** of the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in each of the storage devices **2002a-2002c/2100** (e.g., via the storage engine **2104** and communication system **2108** in that storage device).

As such, in some embodiments, identical copies of the RAID driver telemetry data may be stored in the RAID driver telemetry data section **2202c** of the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in each of the storage devices **2002a-2002c/2100**. However, one of skill in the art in possession of the present

disclosure will also appreciate how RAID driver telemetry data may be stored using the RAID driver telemetry data sections **2202c** of the RAID logical storage subsystems **2202** provided by the storage subsystems **2106** in a plurality of the storage devices **2002a-2002c/2100** (e.g., that RAID driver telemetry data may be stored across multiple RAID driver telemetry data sections **2202c**) while remaining within the scope of the present disclosure as well. Furthermore, one of skill in the art in possession of the present disclosure will also appreciate how the RAID driver telemetry data may be stored in a single RAID driver telemetry data section **2202c** of the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in one of the storage devices **2002a-2002c/2100** while remaining within the scope of the present disclosure as well. As such, prior to the completion of the crash dump operations while in the crash dump mode, any RAID driver telemetry data will be stored in one or more of the storage devices **2002a-2002c**.

The method **1900** then proceeds to decision block **1910** where it is determined whether there is write data in a write-back cache. In an embodiment, at decision block **1910** and in response to entering the crash dump mode, the software RAID engine **204a** may perform write-back cache write data determination operations that may include determining whether the write-back cache subsystem **2001** includes any write data that has not been “flushed” or otherwise written to the storage devices **2002a**, **2002b**, and/or **2002c** (i.e., the write-back cache subsystem **2001** includes a “dirty” cache device). As will be appreciated by one of skill in the art in possession of the present disclosure, write data may have been stored in the write-back cache subsystem **2001** and not yet “flushed” or otherwise written to any of the storage devices **2002a-2002c** prior to that crash, and the software RAID engine **204a** may use any of a variety of techniques to detect and/or otherwise identify that write data in the write-back cache subsystem **2001**.

If, at decision block **1910**, it is determined that there is write data in the write back cache, the method **1900** proceeds to block **1912** where the software RAID subsystem provides the write data in at least one RAID logical storage subsystem provided by at least one storage device in the plurality of storage subsystems. With reference to FIGS. **24A** and **24B**, in an embodiment of block **1912** and in response to detecting write data in the write-back cache subsystem **2001**, the software RAID engine **204a** may perform write-back data storage operations **2400** that may include identifying the storage location for the write-back data in the write-back cache subsystem **2001** (e.g., the storage device **2002b** in this example), and providing that write-back data in the user data section **2202b** of the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in the storage devices **2002b/2100** (e.g., via the storage engine **2104** and communication system **2108** in the storage device **2002b/2100**). As such, prior to the completion of the crash dump operations while in the crash dump mode, any write-back data in the write-back cache subsystem **2001** will be flushed to the one of the storage devices **2002a-2002c** for which it was provided.

If, at decision block **1910**, it is determined that there is no write data in the write back cache, or following block **1912**, the method **1900** proceeds to decision block **1914** where it is determined whether there has been a recovery from the crash. In an embodiment, at decision block **1914**, software RAID engine **204a** in the computing device **2000** may monitor to determine whether the computing device **2000** has been recovered from the crash, and one of skill in the art in possession of the present disclosure will recognize how a

variety of crash recovery operations may be performed to recover the computing device **2000** from that crash such that the computing device **2000** is initialized and again enters the runtime mode described above. As such, following the crash, the software RAID engine **204** may be configured to monitor for the initialization of the software RAID engine **204** (i.e., along with the initialization of the operating system in the computing device **2000** following recovery and entry into the runtime mode) to determine that the computing device **2000** has recovered from the crash. If, at decision block **1914**, it is determined that the computing device **2000** has not recovered from the crash, the method **1900** returns to decision block **1914**. As such, the method **1900** may loop such that the software RAID engine **204a** continues to monitor for the recovery of the computing device **2000** from the crash.

If, at decision block **1914**, it is determined that there has been recovery from the crash, the method **1900** proceeds to block **1916** where the software RAID subsystem retrieves the RAID driver telemetry data from the RAID driver telemetry storage portion of the at least one RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems. With reference to FIGS. **25A** and **25B**, in an embodiment of block **1916** and in response to the computing device **2000** recovering from the crash, the storage engine **2104** in the storage device **2002a/2100** may perform RAID driver telemetry data transmission operations **2500** that include retrieving the RAID driver telemetry data from the RAID driver telemetry data section **2202c** of the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in the storage device **2002a/2100**, and transmitting that RAID driver telemetry data via its communication system **2108** and to the software RAID engine **204a**.

As such, at block **1916**, the software RAID engine **204a** may retrieve the RAID driver telemetry data that was stored during the crash dump mode as described above, and one of skill in the art in possession of the present disclosure will appreciate how the software RAID engine **204a** may provide that RAID driver telemetry data for display to a user (e.g., as part of crash dump information, a crash dump report, a RAID driver report, etc.) in a variety of manners in order to allow the user to analyze the operation of a software RAID driver provided by the software RAID engine **204a** prior to the crash. For example, at block **1916**, the software RAID engine **204a** may parse and process the RAID driver telemetry data to provide it in a human-readable format, and then provide that human-readable RAID driver telemetry data for display to a user.

As will be appreciated by one of skill in the art in possession of the present disclosure, the storage of the RAID driver telemetry data in the RAID logical storage subsystem **2202** provided by the storage subsystem **2106** in the storage device **2002a/2100** as part of the crash dump operations while in crash dump mode allows that RAID driver telemetry data to be retrieved separately from the crash dump data stored during those crash dump operations, thus allowing for analysis and debugging of software RAID driver issues without the difficulties associated with transferring and analyzing the relatively large amount of crash dump data stored during the full crash dumps described above.

Thus, systems and methods have been described that reserve RAID driver telemetry portions of RAID logical storage subsystem in storage devices for use in storing RAID driver telemetry data in the event of a crash. For example, the crash-based RAID data storage system of the present disclosure may include a chassis housing storage

31

subsystems coupled to an operating system and a software RAID subsystem. During a runtime mode of the operating system, the software RAID subsystem reserves a RAID driver telemetry storage portion of a RAID logical storage subsystem provided by at least one storage device in the storage subsystems. In response to the operating system entering a crash dump mode from the runtime mode, the software RAID subsystem initializes respective controller devices in each of the storage subsystems, and during the crash dump mode of the operating system the software RAID subsystem stores RAID driver telemetry data generated by the software RAID subsystem in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device. As such, RAID driver telemetry data that would otherwise be lost in response to a crash is saved and available for use during post-recovery from a crash to, for example, debug a software RAID driver provided by the software RAID subsystem.

Although illustrative embodiments have been shown and described, a wide range of modification, change and substitution is contemplated in the foregoing disclosure and in some instances, some features of the embodiments may be employed without a corresponding use of other features. Accordingly, it is appropriate that the appended claims be construed broadly and in a manner consistent with the scope of the embodiments disclosed herein.

What is claimed is:

1. A crash-based Redundant Array of Independent Disk (RAID) data storage system, comprising:
 - a chassis;
 - a plurality of storage subsystems that are housed in the chassis;
 - an operating system that is provided in the chassis and coupled to the plurality of storage subsystems; and
 - a software Redundant Array of Independent Disk (RAID) subsystem that is provided in the chassis, coupled to the plurality of storage subsystems, and configured to:
 - reserve, during a runtime mode of the operating system, a RAID driver telemetry storage portion of a RAID logical storage subsystem provided by at least one storage device in the plurality of storage subsystems;
 - initialize, in response to the operating system entering a crash dump mode from the runtime mode, respective controller devices in each of the plurality of storage subsystems; and
 - store, in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the operating system, RAID driver telemetry data generated by the software RAID subsystem.
2. The system of claim 1, wherein the software RAID subsystem is configured to:
 - reserve, during the runtime mode of the operating system, a respective RAID driver telemetry storage portion of a respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems; and
 - store, in the respective RAID driver telemetry storage portion of the respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems using the respective controller device for that storage device and during the crash dump mode of the operating system, respective

32

copies of the RAID driver telemetry data generated by the software RAID subsystem.

3. The system of claim 1, wherein the software RAID subsystem is configured to:

- present, to the operating system during the runtime mode of the operating system, a primary controller device that is included in one of the plurality of storage subsystems as being connected to a logical storage device that includes the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems; and

- store, in the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems and outside of the RAID driver telemetry storage portion of the RAID logical storage subsystem during the runtime mode of the operating system, user data transmitted by the operating system to the primary controller device.

4. The system of claim 1, wherein the software RAID subsystem is configured to:

- retrieve, in response to the operating system entering the runtime mode following the crash dump mode and from the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems, the RAID driver telemetry data; and
- provide, during the runtime mode of the operating system following the crash dump mode, the RAID driver telemetry data for display to a user.

5. The system of claim 1, further comprising:

- a write-back cache subsystem included in the software RAID subsystem, wherein the software RAID subsystem is configured to:

- determine, in response to the operating system entering the crash dump mode from the runtime mode, whether the write-back cache subsystem includes write data that has not been provided on a storage device in any of the plurality of storage subsystems.

6. The system of claim 5, wherein the software RAID subsystem is configured to:

- provide, during the crash dump mode of the operating system and in response to determining that the write-back cache subsystem includes the write data that has not been provided on a storage device in any of the plurality of storage subsystems, the write data on at least one storage device in the plurality of storage subsystems.

7. The system of claim 1, wherein the operating system is configured to

- store, outside of the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the operating system, crash dump data.

8. An Information Handling System (IHS), comprising:
 - a processing system; and

- a memory system that is coupled to the processing system and that includes instructions that, when executed by the processing system, cause the processing system to provide a software Redundant Array of Independent Disk (RAID) engine that is configured to:

- reserve, during a runtime mode of an operating system that is coupled to the processing system, a RAID driver telemetry storage portion of a RAID logical storage subsystem that is provided by at least one

33

storage device in a plurality of storage subsystems that are coupled to the processing system;

initialize, in response to the operating system entering a crash dump mode from the runtime mode, respective controller devices in each of the plurality of storage subsystems; and

store, in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the operating system, RAID driver telemetry data generated by the software RAID subsystem.

9. The IHS of claim 8, wherein the software RAID engine is configured to:

reserve, during the runtime mode of the operating system, a respective RAID driver telemetry storage portion of a respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems; and

store, in the respective RAID driver telemetry storage portion of the respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems using the respective controller device for that storage device and during the crash dump mode of the operating system, respective copies of the RAID driver telemetry data generated by the software RAID subsystem.

10. The IHS of claim 8, wherein the software RAID engine is configured to:

present, to the operating system during the runtime mode of the operating system, a primary controller device that is included in one of the plurality of storage subsystems as being connected to a logical storage device that includes the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems; and

store, in the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems and outside of the RAID driver telemetry storage portion of the RAID logical storage subsystem during the runtime mode of the operating system, user data transmitted by the operating system to the primary controller device.

11. The IHS of claim 8, wherein the software RAID engine is configured to:

retrieve, in response to the operating system entering the runtime mode following the crash dump mode and from the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems, the RAID driver telemetry data; and

provide, during the runtime mode of the operating system following the crash dump mode, the RAID driver telemetry data for display to a user.

12. The IHS of claim 8, wherein the software RAID engine is configured to:

determine, in response to the operating system entering the crash dump mode from the runtime mode, whether a write-back cache subsystem that is coupled to the processing system includes write data that has not been provided on a storage device in any of the plurality of storage subsystems.

13. The IHS of claim 12, wherein the software RAID engine is configured to:

provide, during the crash dump mode of the operating system and in response to determining that the write-

34

back cache subsystem includes the write data that has not been provided on a storage device in any of the plurality of storage subsystems, the write data on at least one storage device in the plurality of storage subsystems.

14. A method for storing Redundant Array of Independent Disk (RAID) data in response to a crash, comprising:

reserving, by a software Redundant Array of Independent Disk (RAID) subsystem during a runtime mode of an operating system that is coupled to the software RAID subsystem, a RAID driver telemetry storage portion of a RAID logical storage subsystem that is provided by at least one storage device in a plurality of storage subsystems that are coupled to the software RAID subsystem;

initializing, by the software RAID subsystem in response to the operating system entering a crash dump mode from the runtime mode, respective controller devices in each of the plurality of storage subsystems; and

storing, by the software RAID subsystem in the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the operating system, RAID driver telemetry data generated by the software RAID subsystem.

15. The method of claim 14, further comprising:

reserving, by the software RAID subsystem during the runtime mode of the operating system, a respective RAID driver telemetry storage portion of a respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems; and

storing, by the software RAID subsystem in the respective RAID driver telemetry storage portion of the respective RAID logical storage subsystem provided by each storage device included in the plurality of storage subsystems using the respective controller device for that storage device and during the crash dump mode of the operating system, respective copies of the RAID driver telemetry data generated by the software RAID subsystem.

16. The method of claim 14, further comprising:

presenting, by the software RAID subsystem to the operating system during the runtime mode of the operating system, a primary controller device that is included in one of the plurality of storage subsystems as being connected to a logical storage device that includes the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems; and

storing, by the software RAID subsystem in the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems and outside of the RAID driver telemetry storage portion of the RAID logical storage subsystem during the runtime mode of the operating system, user data transmitted by the operating system to the primary controller device.

17. The method of claim 14, further comprising:

retrieving, by the software RAID subsystem in response to the operating system entering the runtime mode following the crash dump mode and from the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device in the plurality of storage subsystems, the RAID driver telemetry data; and

providing, by the software RAID subsystem during the runtime mode of the operating system following the crash dump mode, the RAID driver telemetry data for display to a user.

18. The method of claim **14**, further comprising: 5
determining, by the software RAID subsystem in response to the operating system entering the crash dump mode from the runtime mode, whether a write-back cache subsystem includes write data that has not been provided on a storage device in any of the 10
plurality of storage subsystems.

19. The method of claim **18**, further comprising:
providing, by the software RAID subsystem during the crash dump mode of the operating system and in response to determining that the write-back cache sub- 15
system includes the write data that has not been provided on a storage device in any of the plurality of storage subsystems, the write data on at least one storage device in the plurality of storage subsystems.

20. The method of claim **14**, further comprising: 20
storing, the operating system outside of the RAID driver telemetry storage portion of the RAID logical storage subsystem provided by the at least one storage device using the respective controller device for that storage device and during the crash dump mode of the oper- 25
ating system, crash dump data.

* * * * *